

Less stress, better scores, same learning: The dissociation of performance and learning in AI-supported programming education

Patrick Bassner*, Ben Lenk-Ostendorf, Ramona Beinstingel, Tobias Wasner, Stephan Krusche

Technical University of Munich, Boltzmannstr. 3, 85748 Garching, Germany

ARTICLE INFO

Keywords:

Artificial intelligence
Educational technology
Cognitive load theory
Scaffolded learning
Large language models
Student motivation

ABSTRACT

Introduction: Generative AI is reshaping programming education, yet its effects on conceptual learning, intrinsic motivation, and cognitive load remain unclear. This study tests whether assistance deepens understanding or primarily boosts task completion, and how scaffolded versus answer-giving designs matter.

Objectives: This study compares performance, learning, cognitive load, frustration, and motivation across three AI support types, and examines students' perceptions.

Methods: A three-arm randomized controlled trial was conducted in an introductory programming (CS1) course at TUM (N = 275). Participants completed a 90-minute exercise on concurrency, implementing a parallel sum with threading in one of three conditions: (1) *Iris*, a scaffolded tutor providing calibrated hints while withholding full solutions; (2) *ChatGPT*, unrestricted assistance that can provide complete solutions; (3) no-AI control using traditional web resources. Pre- and post-knowledge tests and a code comprehension task measured learning, while auto-graded test coverage measured performance. Validated scales captured intrinsic, germane, and extraneous cognitive load, frustration, and intrinsic motivation.

Results: Both AI groups achieved substantially higher exercise scores than the control group, with distinct distributions: *ChatGPT* users clustered at high scores, control participants at low scores, and *Iris* users spread across the full range. Despite these performance gains, neither AI condition produced greater pre-post knowledge gains or code-comprehension advantages. Both AI groups reported lower frustration and reduced extraneous and germane load than the control group, while intrinsic load did not differ. Only *Iris* increased intrinsic motivation. Students rated *ChatGPT* as easier to use and more helpful.

Conclusion: In this setting, generative AI acted primarily as a performance aid rather than a learning enhancer. Scaffolded, hint-first design preserved motivational benefits, whereas AI providing unrestricted solutions encouraged a "comfort trap" where students' preferences misaligned with pedagogical effectiveness. These findings motivate scaffolded AI integration and assessment designs resilient to environments where performance no longer reliably tracks understanding.

1. Introduction

Powerful generative artificial intelligence (AI) systems such as OpenAI's *ChatGPT* or *GitHub Copilot* have upended programming classrooms. Within months of its public release, millions of learners began pasting homework prompts into large-language-model chat windows and receiving syntactically flawless code in return. Universities reacted swiftly—some banning AI outright, others embracing it—yet empirical evidence of how such tools actually influence learning remains scarce. The debate is sharply polarized: critics argue that omnipresent code generators will seduce novices into outsourcing cognitive effort, thereby

stunting genuine understanding, whereas proponents highlight AI's potential to lower frustration, scaffold novices through complex syntax, and potentially help students access expert-level feedback (Liffiton et al., 2023; Peng et al., 2023).

Programming education is a particularly revealing domain for studying these tensions because introductory programming courses (CS1) routinely report high attrition, steep cognitive load, and pronounced student frustration (Lahtinen et al., 2005). Novices must simultaneously grapple with abstract problem decomposition, unfamiliar syntax, and incomprehensible compiler errors—challenges that AI might alleviate by

* Corresponding author.

Email addresses: bassner@tum.de (P. Bassner), ben.lenk-ostendorf@tum.de (B. Lenk-Ostendorf), ramona.beinstingel@tum.de (R. Beinstingel), tobias.wasner@tum.de (T. Wasner), krusche@tum.de (S. Krusche).

<https://doi.org/10.1016/j.caeai.2025.100537>

Received 31 August 2025; Received in revised form 5 December 2025; Accepted 19 December 2025

Available online 24 December 2025

2666-920X/© 2025 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

offering just-in-time hints or outright solutions (Gardella et al., 2024; Huang et al., 2025). Yet the very convenience of immediate answers could short-circuit the productive struggle theorized to foster robust mental models (Baker et al., 2004; Bjork et al., 1994).

This raises a fundamental question: Can AI tools reduce *unproductive* frustration—the extraneous cognitive load imposed by confusing syntax errors and opaque compiler messages—while preserving the *productive* challenge that builds robust mental models? Or does convenient access to solutions eliminate both, creating an illusion of competence?

Recent evidence suggests these concerns are well-founded. A large-scale study of high school mathematics students found that those using generative AI without pedagogical guardrails performed significantly worse on subsequent assessments when AI support was removed, suggesting that unrestricted AI access can foster dependency rather than genuine skill development (Bastani et al., 2025). Beyond learning outcomes, AI assistance introduces professional risks: novice programmers often struggle to validate AI-generated code, accepting flawed suggestions without adequate verification (Ferdowsi et al., 2024). This validation gap raises concerns not only about academic integrity but also about the development of critical professional competencies.

These educational concerns intersect with evolving workforce demands. Recent research demonstrates that generative AI tools are fundamentally reshaping software development practices, with developers needing to master evolving prompting techniques to work effectively (France, 2024). Software engineering educators argue that curricula must evolve to prepare graduates who can effectively leverage AI tools while maintaining the critical judgment necessary to evaluate their outputs (Kirova et al., 2024). This shift suggests that programming education must balance AI fluency with foundational competencies in validation, debugging, and critical assessment of machine-generated solutions.

Hidden within this debate lies a more subtle challenge: students themselves may be poor judges of what helps them learn. Research consistently shows that learners prefer methods that feel easier, even when those methods produce inferior learning outcomes (Deslauriers et al., 2019). If AI tools make programming feel effortless, will students gravitate toward the very features that undermine their education? Moreover, not all AI assistance is created equal. The design choices embedded in these tools—whether they provide complete solutions or calibrated hints, whether they preserve student autonomy or encourage dependency—may fundamentally shape their educational impact (Jennings & Muldner, 2021; Wood et al., 1976).

Against this backdrop, this study investigates a potential paradox in AI-supported programming education: tools that improve task performance may leave conceptual understanding entirely unchanged. The investigation examines not only whether students succeed more with AI assistance, but whether this success translates to genuine learning—and whether different AI designs can thread the needle between support and challenge. This study addresses the following research questions:

1. How do different AI support types affect students' exercise performance versus their actual learning outcomes?
2. What are the differential cognitive-load implications of direct-solution provision compared with scaffolded AI support?
3. How do these AI modalities influence student motivation and frustration during CS1 programming tasks?

To answer these questions, a three-group randomized controlled trial (RCT) examined $N = 275$ students enrolled in a CS1 course at the TUM. Participants completed a 90-minute, auto-graded programming task covering concurrent programming concepts using Java. Depending on random assignment, they either (1) interacted with *Iris*, a context-aware tutor that provides incremental hints but withholds full solutions to preserve productive struggle (Bassner et al., 2024); (2) used *ChatGPT*, receiving unrestricted answers on demand; or (3) had no AI access and

relied on conventional web resources. Pre- and post-tests measured conceptual knowledge, while validated surveys captured cognitive load, intrinsic motivation, and frustration.

The findings reveal surprising patterns that challenge assumptions on both sides of the AI debate. While both AI-assisted groups enabled students to achieve higher exercise scores, this study found no statistically significant differences in learning gains compared to traditional resources. More intriguingly, only the *Iris* group enhanced student motivation, while the *ChatGPT* group—despite being perceived as more helpful—showed no motivational benefit. These results suggest that the educational impact of AI depends critically on its pedagogical design, not merely its technical capabilities.

The remainder of this article is organized as follows. Section 2 situates this study within prior research on generative AI in programming education, cognitive load, motivation, and scaffolding. Section 3 details the tutor's architecture and calibrated-assistance paradigm. Section 4 describes the randomized controlled trial, experimental conditions, materials, and measures. Section 5 explains preprocessing, exclusion criteria, composite-score construction, and statistical methods. Section 6 presents performance, learning, cognitive-load, and motivational outcomes, including a direct comparison of the two AI tools. Section 7 synthesizes the results into key takeaways. Section 8 interprets these findings and outlines implications for assessment and AI-tutor design. Section 9 examines construct, internal, external, and reliability considerations. Finally, Section 10 summarizes contributions and future directions.

2. Background and related work

Educational chatbots have been deployed across diverse educational contexts (Okonkwo & Ade-Ibijola, 2021), yet their application in programming education presents a persistent tension: reducing unproductive frustration while preserving productive cognitive effort. This section examines how AI tutors might navigate this tension. We first establish theoretical foundations from Cognitive Load Theory, Self-Determination Theory, and scaffolding research that frame the core design challenge. We then review empirical evidence on AI in programming education, tracing findings on performance, learning, and affect before examining scaffolding as a design response. Finally, we synthesize the methodological gaps this literature reveals and position the study's contributions.

2.1. Theoretical foundations: cognition, motivation, and support

Programming education research increasingly draws upon cognitive and motivational frameworks to design and evaluate AI-powered interventions. Three theoretical perspectives provide essential foundations for understanding how AI tutors influence learning processes, motivation, and instructional design: Cognitive Load Theory, Self-Determination Theory, and the concept of scaffolding within the Zone of Proximal Development. After establishing these foundations, we examine two cross-cutting challenges—the assistance dilemma and maladaptive help-seeking—that emerge when applying these theories to instructional design.

Cognitive Load Theory (CLT) distinguishes three types of cognitive load that learners experience during instruction (Paas et al., 2003; Sweller et al., 1998). *Intrinsic load* arises from the material's inherent element interactivity—the extent to which information elements must be processed simultaneously in working memory. *Germane load* represents productive cognitive effort directed toward schema construction and learning. *Extraneous load* refers to wasted mental effort imposed by poor instructional design or presentation. Effective instruction seeks to reduce extraneous load, manage intrinsic load within learners' working memory capacity, and maximize germane load to promote deep learning. In the context of AI tutors, this framework raises a critical design tension: while AI support can reduce extraneous load by providing timely assistance and clarifying misconceptions, it may also inadvertently undermine learning through multiple mechanisms.

Cognitive offloading—delegating mental work to external tools—creates a performance-memory trade-off: while offloading improves immediate task performance, it diminishes subsequent memory for the offloaded information (Grinschgl et al., 2021). AI tutors providing direct answers may thus enable task completion while bypassing the productive cognitive struggle essential for durable learning.

Which forms of cognitive effort actually benefit learning? Not all difficulty is detrimental. Bjork's framework of *desirable difficulties* identifies specific instructional manipulations that temporarily impair performance during acquisition but enhance long-term retention and transfer (Bjork et al., 1994). Crucially for AI tutoring, the *generation effect*—requiring learners to actively produce answers rather than passively receive them (Slamecka & Graf, 1978)—demonstrates that active production consistently yields superior retention compared to passive reception, though the precise cognitive mechanisms remain debated in the literature. In contrast, difficulties become *undesirable* when learners lack the background knowledge or skills to respond to them successfully, wasting cognitive resources without contributing to schema construction (Bjork & Bjork, 2011). This distinction maps directly onto CLT's germane versus extraneous load dichotomy: effective AI support should eliminate undesirable difficulties while preserving the generative processing that consolidates learning. Research on affective dynamics suggests that temporary confusion can signal productive cognitive engagement when resolved through effort, but prolonged confusion without resolution degrades into frustration and disengagement (D'Mello & Graesser, 2012).

Self-Determination Theory (SDT) offers a complementary motivational perspective, positing that intrinsic motivation—characterized by interest, enjoyment, and sustained engagement—arises when three basic psychological needs are satisfied: autonomy (experiencing a sense of volition and choice), competence (feeling effective and capable), and relatedness (experiencing connection with others) (Ryan & Deci, 2000). Autonomy-supportive learning environments that provide meaningful choices and acknowledge learners' perspectives foster intrinsic motivation, whereas controlling environments that pressure learners or undermine their agency tend to diminish it (Deci & Ryan, 1987). Applied to AI tutors in programming education, SDT suggests that both hint-based systems and unrestricted answer-providing systems may support learners' sense of competence by enabling task completion, yet they differ substantially in their impact on autonomy: hint-based support preserves learner agency by requiring active problem-solving, while direct answer provision may undermine autonomy by removing the challenge and decision-making opportunities that sustain engagement. From this perspective, excessive assistance may diminish intrinsic motivation even when performance improves.

The concept of scaffolding, rooted in Vygotsky's Zone of Proximal Development (ZPD), provides a third foundational perspective (Vygotsky, 1978). The ZPD represents the gap between what learners can accomplish independently and what they can achieve with appropriate support. Scaffolding refers to temporary, calibrated assistance that enables learners to succeed within their ZPD. Wood et al. (1976) introduced the scaffolding metaphor to describe tutorial support that controls task elements initially beyond the learner's capacity, enabling them to focus on manageable components. They identified six key tutoring functions: recruiting learner interest, reducing degrees of freedom in the task, maintaining direction toward goals, marking critical features, controlling frustration, and demonstrating solutions when learners can recognize them. Building on this foundation, subsequent scaffolding research has converged on three key characteristics: effective support should be contingent (responsive to learner needs), graduated (adjusting support levels as competence increases), and eventually faded as learners achieve independence (van de Pol et al., 2010). This fading requirement connects to CLT: instructional guidance that benefits novices can become counterproductive for more advanced learners when external support overlaps with existing knowledge, forcing learners to expend cognitive resources reconciling redundant

information rather than constructing new schemas (Kalyuga, 2007). In AI-supported programming education, this framework distinguishes between systems that provide genuine scaffolding—adaptive hints that guide without revealing solutions—and those that simply deliver answers, bypassing the ZPD entirely. The pedagogical challenge lies in designing AI tutors that maintain learners within their ZPD through appropriately calibrated support, rather than either abandoning them without guidance or completing tasks on their behalf.

These three frameworks suggest a potential pedagogical tension: while AI assistance may reduce extraneous load and help learners work within their ZPD, it risks undermining learning if it eliminates the germane cognitive processing essential for schema construction. Two cross-cutting challenges complicate this picture: the assistance dilemma and maladaptive help-seeking behaviors.

Providing instructional assistance introduces what Koedinger and Aleven term the “assistance dilemma” (Koedinger & Aleven, 2007): a fundamental open problem in instructional science concerning how to balance information giving and withholding. Too little assistance causes unproductive frustration and floundering, whereas too much can lead to shallow processing that bypasses the cognitive engagement necessary for learning. The optimal level of assistance depends on both learner characteristics (prior knowledge, expertise level) and task demands (complexity, cognitive load) (Van Merriënboer & Sweller, 2005). For instance, a student struggling with thread synchronization might benefit from a hint identifying the need for shared state protection, whereas receiving a complete synchronized block implementation would eliminate the productive cognitive processing required to understand *why* synchronization is necessary and *when* to apply it. Empirical research on intelligent tutoring systems demonstrates that moderate support—hints and guiding questions rather than complete solutions—maximizes learning gains, whereas maximal assistance often proves counterproductive (Jennings & Muldner, 2021). Generative AI introduces a qualitatively different context: LLMs can generate complete solutions effortlessly (Peng et al., 2023), substantially changing the support landscape. The pedagogical question is therefore not whether AI *can* provide help, but how much help is optimal for learning. Whether principles from rule-based intelligent tutoring systems generalize to LLM contexts remains an open empirical question.

Beyond system design, learner behavior matters: students may not use help appropriately. Aleven et al. identify a taxonomy of maladaptive help-seeking behaviors: *help abuse*, whereby students request hints without genuinely processing their content, and *help avoidance*, whereby students persist in unproductive struggle without seeking assistance when needed (Aleven et al., 2016). Help abuse typically reflects metacognitive difficulties in assessing one's own understanding, leading students to request hints preemptively rather than attempting problems independently (Roll et al., 2011). Help avoidance, in contrast, often stems from performance-avoidant goals—concerns about demonstrating incompetence relative to others (Karabenick, 2004; Ryan & Pintrich, 1997). Related to help abuse, Baker et al. demonstrated that students often “game the system” by systematically exploiting regularities in tutoring software—requesting help without genuine engagement, clicking through hint sequences without reading them, or intentionally entering wrong answers to trigger feedback (Baker et al., 2004).

In sum, the challenge is not whether to provide support, but rather *how* to calibrate that support so it preserves the productive cognitive engagement necessary for genuine expertise rather than performance that evaporates when support is withdrawn.

2.2. Generative AI in programming education: empirical evidence

Recent empirical work has begun examining how generative AI affects programming education, though the evidence base remains emerging. We first review studies examining performance and affective outcomes, then turn to scaffolding as a design response.

2.2.1. Performance and affect

Peng et al. conducted a randomized controlled trial with 95 professional developers recruited through Upwork, tasking participants with implementing an HTTP server in JavaScript as quickly as possible (Peng et al., 2023). Developers using GitHub Copilot completed the task 56 % faster, with modestly higher (though not statistically significant) success rates. The study examined a single, standardized task rather than collaborative professional contexts, measured only completion speed and success without assessing code quality or maintainability, and explicitly did not evaluate learning outcomes. This exemplifies a broader gap in AI-assisted programming research: productivity gains are well-documented, yet learning effects remain largely unexplored, and professional developers differ substantially from novice learners in both prior knowledge and learning goals.

Gardella et al. examined 17 novice programmers completing HumanEval tasks with and without GitHub Copilot under time pressure (Gardella et al., 2024). Copilot significantly improved performance and reduced workload as measured by NASA-TLX, yet no significant effects emerged for emotion or immediate self-efficacy. Drawing on Cognitive Load Theory, the study revealed that self-efficacy improved only on the second AI trial, suggesting competence develops through repeated experience rather than immediate tool access. The small sample, controlled laboratory conditions, brief exposure, and self-efficacy operationalized through self-grading rather than validated scales constrain generalizability to authentic classroom contexts. Nevertheless, the null affective findings suggest that reducing workload alone is insufficient without addressing learners' sense of capability.

Shihab et al. tasked 10 undergraduate computer science students with brownfield programming—adding features to unfamiliar legacy web applications—with and without GitHub Copilot (Shihab et al., 2025). Students using Copilot achieved substantially faster task completion and more solution progress, while spending less time writing code and conducting web searches. Yet students expressed concerns about not understanding how or why Copilot suggestions worked, indicating a comprehension-performance gap. Validity threats include Copilot's auto-updates creating inconsistent experimental conditions, a non-counterbalanced design leaving learning effects unmitigated, and a small sample of high-achieving students from a single institution.

Fan et al. examined AI-assisted pair programming's impact on intrinsic motivation among 234 undergraduate students, comparing AI assistance (GPT-3.5 Turbo, Claude 3 Opus), human pairing, and individual programming (Fan et al., 2025). AI-assisted groups demonstrated significantly increased intrinsic motivation, reduced programming anxiety compared to individual programming, and significantly outperformed individual programming ($p < .01$), with performance comparable to human-human pair programming. Grounded in Self-Determination Theory, the findings attribute motivational gains to enhanced autonomy (immediate feedback enabling independent problem exploration) and competence (successful interactions building confidence). Human pairing fostered significantly higher social presence and collaborative depth than AI assistance, suggesting limitations in fulfilling the relatedness dimension of SDT. The quasi-experimental design introduces potential confounding variables, and generalizability may be constrained by the sample's institutional and cultural context.

These studies establish that AI assistance accelerates task completion and can enhance motivation, but whether such gains translate to learning remains unclear.

2.2.2. Scaffolding as a design response

Research on intelligent tutoring systems provides foundational evidence on optimal scaffolding levels. Jennings and Muldner investigated scaffolding in code-tracing instruction through a controlled experiment with 97 undergraduate students, manipulating interface scaffolding and instructional sequencing in a purpose-built intelligent tutoring system (Jennings & Muldner, 2021). Their multi-layered analysis, combining inferential statistics with unsupervised clustering of learner behaviors,

revealed that for students who demonstrated learning gains, optimal assistance fell at intermediate levels—providing either worked examples or high interface scaffolding, but not both or neither. Critically, the study identified substantial individual differences in optimal assistance requirements across learner subgroups, challenging one-size-fits-all approaches to instructional support. These findings suggest that excessive scaffolding may encourage passive rather than constructive engagement, while insufficient support leaves learners struggling unproductively.

The most striking empirical evidence for how scaffolding shapes AI-assisted learning comes from Bastani et al., whose pre-registered randomized controlled trial enrolled nearly 1000 high school mathematics students across four 90-minute sessions, comparing unrestricted GPT-4, scaffolded graduated hints, and textbook-only controls (Bastani et al., 2025). While AI access substantially improved practice performance, with the scaffolded tutor showing larger gains than unrestricted AI, subsequent examinations administered *without* AI access revealed a striking result: unrestricted-AI students performed *worse* than controls who never accessed AI, suggesting over-reliance impeded skill acquisition. The scaffolded condition largely mitigated these negative effects, though positive transfer to independent performance remained elusive. The scaffolded system provided hints rather than direct answers, requiring students to interpret guidance and construct their own solutions rather than receiving complete responses immediately. In contrast, unrestricted access eliminated this regulatory requirement, allowing students to obtain complete solutions without engaging in the diagnostic self-assessment that signals knowledge gaps. This study provides the strongest causal evidence that scaffolding mitigates AI dependency, though whether these findings generalize from symbolic mathematics to programming—where debugging concurrent code requires qualitatively different cognitive processes—remains untested.

Several LLM-based tutoring systems have operationalized scaffolding principles for programming education. Huang et al. provide preliminary evidence from programming education, comparing a scaffolded LLM tutor (CodeFlow Assistant) against traditional teaching assistant support in a first-year Python course using a quasi-experimental design (Huang et al., 2025). The CodeFlow Assistant implemented four-tier graduated scaffolding (flowcharts, cloze coding exercises, basic solutions, and advanced solutions) calibrating support to student needs. The AI group achieved significantly higher coding performance, yet showed only *comparable* improvements in conceptual understanding on assessments administered without AI—a dissociation between performance and learning similar to that observed in Bastani et al.'s mathematics study. Critically, cognitive load analysis revealed that the scaffolded AI significantly enhanced germane load, suggesting greater allocation of working memory resources to productive learning activities, though effects on intrinsic and extraneous load did not reach significance. The intervention also significantly improved self-regulated learning abilities, including intrinsic goal orientation, task value, and metacognitive self-regulation. The quasi-experimental design with non-random group assignment limits causal inference despite statistical controls, and the comparison to human TAs rather than unrestricted AI leaves open whether scaffolding constraints specifically—versus structured assistance generally—drove the observed benefits.

Liffiton et al. deployed CodeHelp, an LLM-powered assistant with pedagogical guardrails that provide hints without revealing solutions, across a 12-week semester with 52 students in an introductory programming course (Liffiton et al., 2023). The system employed a multi-prompt workflow designed to guide students toward solutions while withholding direct answers. Students reported that CodeHelp was well-received for its availability and ability to help resolve programming errors. However, the single-group observational design relied on self-reported perceptions rather than objective learning outcome measurements, and the absence of a control condition precludes causal inference about whether the scaffolding constraints specifically—versus AI assistance generally—produced observed benefits.

Kazemitabaar et al. deployed CodeAid at larger scale, offering five scaffolded features including interactive code explanation and error annotation without direct solutions, across a 12-week C programming course with 700 students (Kazemitabaar et al., 2024). Analysis of thousands of usage instances, weekly surveys, and student interviews revealed that most responses were technically correct, with accuracy varying by feature type—higher for conceptual questions than for debugging assistance. The system successfully avoided revealing direct code solutions, instead generating pseudocode explanations and highlighting errors with suggested fixes. The classroom evaluation lacked a control group without AI access, preventing comparison against traditional instruction or assessment of whether scaffolding constraints specifically improve learning beyond what unrestricted AI might provide.

2.3. Research gap and present study

The studies reviewed above converge on several findings while revealing critical methodological gaps. Performance benefits of AI assistance are well-established: Peng et al. (2023), Gardella et al. (2024), and Shihab et al. (2025) consistently document faster task completion and higher success rates with AI tools. Yet these same studies acknowledge that they did not assess learning outcomes, leaving open whether productivity gains translate to knowledge acquisition.

The scaffolding literature addresses this gap, but with important limitations. Bastani et al. (2025) mathematics RCT demonstrated that unrestricted AI access *harm*ed learning relative to no-AI controls, while scaffolded hints largely mitigated these negative effects. Huang et al. (2025) provide preliminary evidence from programming, finding that scaffolded AI yields performance gains but only *comparable* conceptual understanding—yet their comparison to human TAs rather than unrestricted AI cannot isolate scaffolding effects. Within programming, systems like CodeHelp and CodeAid show promise: students report positive experiences and the systems successfully withhold direct solutions. Yet as Liffiton et al. (2023) and Kazemitabaar et al. (2024) explicitly note, their single-group deployments without control conditions cannot establish whether observed benefits stem from scaffolding constraints specifically or from AI assistance generally.

Affective outcomes add further complexity. Fan et al. (2025) found that AI assistance increased intrinsic motivation. Gardella et al. (2024) reported null findings for self-efficacy despite performance gains suggesting that reducing workload alone may be insufficient. These divergent findings likely reflect differences in scaffolding design, yet no study has directly compared scaffolded and unrestricted AI on affective outcomes within the same experimental design.

Three specific gaps emerge from this synthesis:

First, no three-arm experimental design in programming education has isolated scaffolding effects by comparing scaffolded AI, unrestricted AI, and no-AI conditions simultaneously. Two-arm designs comparing scaffolded AI against human support (Huang et al., 2025) cannot distinguish scaffolding benefits from AI benefits generally; single-group deployments of scaffolded systems (Kazemitabaar et al., 2024; Liffiton et al., 2023) lack the control conditions needed for causal inference.

Second, existing studies fragment outcome measurement across separate investigations. Performance studies (Gardella et al., 2024; Peng et al., 2023) omit learning assessments; scaffolding studies (Bastani et al., 2025; Huang et al., 2025) provide limited affective measurements; affective studies (Fan et al., 2025) rarely include objective learning outcomes. No single design has measured performance, learning, cognitive load, and motivation comprehensively enough to assess potential tradeoffs.

Third, how scaffolding constraints shape the interplay between performance, cognition, and motivation in programming remains unclear. Existing deployments have focused on usability and student perceptions rather than controlled comparisons of cognitive and motivational outcomes against unrestricted AI. Whether scaffolded AI preserves productive challenge, enhances motivation, or merely reduces

frustration without corresponding learning benefits requires controlled investigation.

The present study addresses these gaps through a three-arm randomized controlled trial comparing scaffolded AI (Iris), unrestricted AI (ChatGPT), and traditional resources (No AI) in programming education. By holding the underlying model constant (GPT-4.0) while varying only the scaffolding constraints, the design tests whether pedagogical guardrails affect learning, cognitive, and motivational outcomes differently than unmediated AI access. Comprehensive outcome measurement—spanning exercise performance, knowledge assessment, code comprehension, cognitive load subscales, frustration, and intrinsic motivation—enables assessment of potential tradeoffs. Perception items allow direct comparison of students' subjective tool evaluations against their objective outcomes.

3. The Iris AI-tutor: system design and pedagogical framework

The AI tutor *Iris*, introduced by Bassner et al. (2024), embodies the scaffolding principles that prior research has identified as critical for effective educational AI. Unlike external chatbots such as *ChatGPT*, *Iris* leverages deep integration with the *Artemis* learning-management system (Krusche & Seitz, 2018) to access rich, real-time context including problem statements, code submissions, build logs, and performance traces. This contextual awareness enables *Iris* to provide situated assistance without requiring students to repeatedly explain their problems (Bassner et al., 2024).

As the primary experimental intervention in this study, *Iris* operationalizes the design principles emerging from prior work through three architectural pillars: **educational scaffolding** via *calibrated hints* that preserve productive struggle, a **context-aware agent** architecture that reasons over LMS data in real time, and a **multimodal retrieval-augmented generation** (RAG) pipeline that grounds explanations in authoritative course artifacts. This experiment employed *Iris* with *OpenAI's GPT-4* as the LLM. The section concludes by contrasting *Iris* with the comparison conditions used in this study.

3.1. Educational scaffolding: calibrated assistance

Scaffolding theory posits that temporary, tailored support can help learners bridge knowledge deficits while still fostering independent problem-solving (Wood et al., 1976). In programming courses, indiscriminately providing full solutions might short-circuit learning and undermine cognitive engagement. *Iris* therefore attempts to preserve *productive struggle* by reducing unproductive frustration while maintaining the productive cognitive effort needed for mastery (Section 2.1). Empirical classroom studies confirm that assistants offering partial guidance outperform those providing complete answers (Kazemitabaar et al., 2024).

Iris modulates help along four ascending tiers of support. At the most basic level, *subtle hints* focus attention on salient code lines or conceptual blind spots without providing explicit solutions. The second tier employs *guiding questions* that provoke reflection and self-discovery, such as asking “How does your loop behave on an empty list?” to encourage students to trace through their logic. Moving up the hierarchy, *high-level conceptual feedback* offers strategic guidance without revealing specific code implementations, helping students understand the broader approach while preserving the challenge of implementation. Finally, at the highest level of support, *generalized examples* illustrate analogous patterns—such as demonstrating generic recursion principles—while deliberately keeping the target solution opaque to maintain productive struggle. This four-tier progression constitutes what the authors term *Calibrated Assistance* (Bassner et al., 2024), offering two key pedagogical advantages. First, it minimizes unproductive frustration while sustaining meaningful cognitive effort; second, it aligns with calls to design AI tutors that strengthen problem-solving skills rather than substitute for them (Denny et al., 2024; Kasneci et al., 2023). By requiring learners to interpret hints and construct their own solutions, this graduated

approach preserves learner autonomy—a core component of intrinsic motivation according to Self-Determination Theory (Ryan & Deci, 2000). The four-tier structure also navigates the assistance dilemma (Koedinger & Alevan, 2007): it aims to provide enough support to prevent unproductive frustration while withholding complete solutions that would short-circuit the productive struggle necessary for schema construction.

3.2. A context-aware, dynamic agent

Iris functions as an LLM-powered agent capable of iterative perception–reasoning–action loops (Xi et al., 2025). Using an ART-style implementation (Paranjape et al., 2023) for automatic multi-step reasoning and tool use, the agent composes multi-step plans and dynamically invokes specialized tools (e.g., code-analysis or lecture-retrieval tools) until it has gathered sufficient evidence to answer. Recent benchmarks show that such agentic workflows outperform fixed pipelines in open-ended problem settings (Rawat et al., 2025; Wu et al., 2025). Unlike external chatbots, *Artemis* embeds *Iris* directly within its framework, granting the tutor secure, automated access to comprehensive contextual information. This integration encompasses problem specifications, including assignment text and grading rubrics, as well as student artifacts such as current code submissions, prior attempts, and error logs. Additionally, *Iris* can access lecture content spanning slide decks and video transcripts, alongside auxiliary resources including FAQs, documentation, and exemplar solutions. Such holistic context allows *Iris* to respond like a human teaching assistant. For instance, upon detecting repeated test failures, the agent can fetch the failing log, locate the relevant lecture snippet, and generate a hint that addresses the learner's conceptual misunderstanding, all without manual copy–paste by the student. By automatically gathering and integrating relevant artifacts, *Iris* reduces the unproductive cognitive demands (Section 2.1) that students would otherwise expend on searching for information, switching between interfaces, or re-explaining their problem context. This streamlining allows learners to focus their mental resources on understanding and solving the core programming challenge, rather than on managing the mechanics of information retrieval.

3.3. Grounding responses in course content: a retrieval-augmented generation pipeline

Although the experiment of this study emphasizes programming-exercise support, *Iris*'s credibility still depends on *grounded* answers. Retrieval-Augmented Generation (RAG) combines the generative fluency of LLMs with the factual reliability of retrieved passages, thereby reducing hallucinations and aligning terminology with course materials (Li et al., 2025; Shan, 2025).

Systematic reviews show that grounding answers in vetted material improves factual accuracy, aligns terminology with course conventions, and provides learners with authoritative follow-up references (Li et al., 2025). Faculty trials of intelligent-agent tutors that combine RAG with LMS context have similarly reported higher trust and learning gains than generic chatbots (Horia Alexandru, 2025).

Iris employs a multi-stage retrieval-augmented generation pipeline optimized for educational contexts. The ingestion phase chunks slide text and video transcripts into searchable segments and embeds them into a vector store. The system automatically captions diagram-only slides to ensure text-based retrieval can access visual information. During query processing, the system enhances student questions through normalization procedures (including clarification and translation) and expansion using HyDE synthetic answers, thereby improving recall of semantically related passages (Gao et al., 2023). The retrieval mechanism employs a hybrid approach that combines dense-vector similarity matching with keyword-based retrieval, then reranks results using a lightweight listwise cross-encoder to optimize top- k precision (Ma et al., 2023). In the final grounded generation phase, the system provides the highest-ranked snippets (typically 3–5) to the LLM alongside an explicit *cite-only* directive, ensuring that generated responses include slide

numbers or timestamps that facilitate both instructor verification and student follow-up references.

In sum, *Iris*'s RAG pipeline ensures that every answer is not only relevant and accurate, but also transparently linked to authoritative course materials. This approach addresses two major challenges of LLM-based tutoring: reducing hallucinations and aligning explanations with instructor intent. By surfacing citations and grounding responses in vetted content, *Iris* enables both students and instructors to trust and verify the AI's guidance.

3.4. Comparison conditions

To contextualize *Iris*'s design, this study compares it against two alternatives that differ systematically in the pedagogical dimensions outlined above.

Unrestricted AI assistance (ChatGPT). General-purpose LLMs like *ChatGPT* represent a class of AI tools that provide direct, unconstrained responses to user queries. Regarding *scaffolding*, these systems lack pedagogical guardrails: when students request complete solutions, the model complies. Observational studies of novice programmers using AI code generation tools report that students frequently accept generated code with minimal critical evaluation, often struggling to explain or modify solutions they did not construct themselves (Prather et al., 2023). This stands in contrast to hint-based systems designed to preserve productive struggle. Regarding *context-awareness*, general-purpose chatbots operate without access to course materials, exercise specifications, or student code unless users manually provide this information—unlike IDE-integrated tools or purpose-built educational assistants. Regarding *grounding*, responses draw on the model's pre-training corpus rather than authoritative course content; students have noted that *ChatGPT* “sometimes used functions that were not used in the course” (Kazemitabaar et al., 2024).

Traditional web resources (No-AI control). Self-directed learning via web searches, documentation, and Q&A sites like *StackOverflow* represents the pre-AI baseline in programming education. Unlike AI assistants, these resources provide no *scaffolding* in the interactive sense—students receive static content rather than adaptive guidance calibrated to their current understanding. There is no *context-awareness*: generic web content cannot adapt to course-specific requirements or individual student progress. *Grounding* varies widely, as web content ranges from authoritative documentation to outdated forum posts, requiring students to independently evaluate source quality. This condition thus tests whether AI assistance provides genuine pedagogical value beyond what motivated learners can achieve through traditional self-directed means.

4. Methodology

This study employed a randomized controlled trial (RCT) to investigate how different AI support modalities affect student performance, learning outcomes, and motivation in programming education. The three-group experimental design compared scaffolded AI support (*Iris*), unrestricted AI assistance (*ChatGPT*), and traditional web-based resources (No-AI control). Fig. 1 visualizes the comprehensive research procedure, which followed an eight-step process spanning several months of preparation and execution. The initial design phase established the experimental framework. Preparation involved developing validated surveys measuring cognitive load, motivation, and learning outcomes, alongside creating the challenging concurrent programming exercise (“Divide and Conquer”) that required students to implement parallel sum calculators using Java's concurrency primitives. A critical preparatory step included comprehensive tutor training with theoretical introduction, complete dry runs, and detailed written protocols to ensure consistent execution across 35 tutorial sessions with 19 different tutors. The experimental execution phase involved running tutorial groups

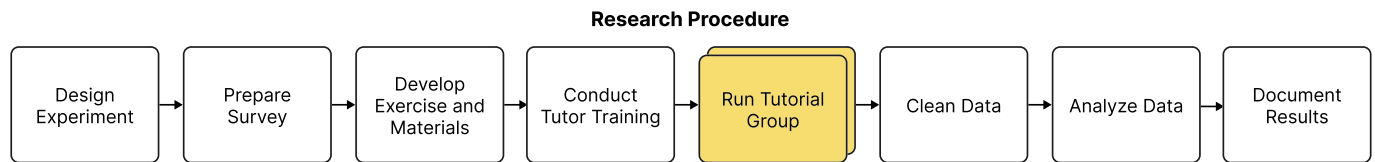


Fig. 1. Overview of the research procedure. The study followed an eight-step process: (1) design experiment, (2) prepare surveys and materials, (3) develop exercise and tutorial, (4) conduct tutor training, (5) run tutorial groups, (6) clean data, (7) analyze data, and (8) document results.

where 452 students initially participated in 90-minute sessions, followed by rigorous data cleaning applying systematic quality controls that resulted in 275 participants with complete, valid data. The analysis phase employed multiple statistical approaches to examine performance, learning gains, cognitive load, and motivational outcomes, culminating in comprehensive documentation of findings. This methodical approach incorporated both objective performance measures and subjective student experiences to provide a holistic understanding of AI's impact on programming education.

4.1. Participants and setting

This experiment took place within an introductory programming (CS1) course at TUM. The course introduces management students to core computer science concepts on a fundamental, practical, and scientific level. Students learn to solve manageable algorithmic problems and independently implement basic applications in Java. The course focuses on teaching essential concepts and models, enabling students to acquire skills transferable to other object-oriented programming languages.

The course is part of the Bachelor's and Master's programs, serving either as a compulsory module for the students' technical focus or as an elective module within the economic-technical curriculum. In the semester under study, 571 Master's and Bachelor's students actively participated in the course, most of whom had little or no prior experience in object-oriented programming. The course awards 12 ECTS credits, corresponding to an expected workload of 360 hours.

The course structure includes regular interactive workshops that introduce and explain fundamental programming concepts and methods using practical examples. Small exercises, such as quizzes and programming exercises, provide additional opportunities for students to test their understanding. Individual feedback and tutor support during the workshop help students assess their progress and clarify any difficulties. These workshops are complemented by 35 weekly on-site tutorial group sessions, each accommodating around 20 students. 19 different tutors facilitated these groups, with most tutors leading two sessions. Tutors include management students who have completed the course and computer science students with experience in a comparable, more advanced course within their studies. Each student attends one tutorial group per week. During these sessions, students work on programming exercises and present their solutions to homework exercises related to the week's topics.

The tutorial group sessions provided an effective setting for conducting the experiment, ensuring alignment with the course structure and encouraging broad student participation. Two main factors guided this decision: students were already familiar with completing tutor exercises within a tutor group, and it reduced participation barriers, as students avoided making additional trips to the university. A total of 452 students initially participated in the experiment; after predefined data-quality controls (see Section 5.1), 275 remained for analysis.

4.2. Experimental conditions

The study randomly assigned participants to one of three experimental conditions using their University IDs as a seed for random assignment. Baseline balance checks on gender, programming experience, and age confirmed group comparability (see Table 3). Neither the

tutors nor the students knew about their assigned condition until the start of the exercise.

Iris group (IRIS): scaffolded AI support. Students used only *Iris* (see Section 3) within *Artemis* based on *OpenAI's GPT-4.0* for the experiment, avoiding the web, *ChatGPT*, or other AI assistants. They accessed *Iris* through an integrated chat window within the learning platform, a feature they were already familiar with from previous use during the semester. *Iris* provided hints and tips but did not deliver direct solutions. By submitting their code, they enabled *Iris* to view it and offer feedback. In addition, *Iris* had access to the exercise description and course materials. The presence of the *Iris* logo on the exercise page indicated their group assignment.

ChatGPT group (GPT): unrestricted AI assistance. Students used only *ChatGPT* within *Artemis* based on *OpenAI's GPT-4.0* for the experiment, excluding the web, *Iris*, or other AI assistants. They accessed *ChatGPT* through the same integrated chat window where *Iris* would typically appear. Unlike *Iris*, *ChatGPT* provided more direct and detailed responses. In general, contrary to *Iris*, this AI assistant would follow all requests of the user, even if they included requests for the full solution. However, *ChatGPT* did not have access to any course-specific context, the exercise description, the student's code, or other situation-specific context. The *OpenAI* logo on the exercise page signified their group assignment.

No-AI control group (NOAI): traditional resources. Students relied on lecture slides and web resources, such as StackOverflow, Google, or YouTube, to complete the exercise. They were explicitly prohibited from using any AI tools, including *Iris*, *ChatGPT*, or similar assistants. Tutors actively moved through the room to monitor compliance with this rule. The absence of both the *Iris* and *OpenAI* logos on the exercise page indicated their group assignment.

4.3. Programming exercise selection and rationale

The programming exercise needed to balance several competing requirements: sufficient complexity to minimize trivial AI solutions, alignment with course content, and feasibility within a 90-minute time limit. At the time of the experiment, the course schedule had reached its final topics, with only two exercise weeks remaining: recursion and concurrency.

The study selected the concurrency exercise, titled "Divide and Conquer,"¹ which required students to implement a parallel sum calculator. The task involved splitting the computation of summing numbers from 1 to N across multiple threads using Java's concurrency primitives: *ExecutorService*, *Callable*, *Future*, *Consumer*, and *SubmissionPublisher*. Students needed to initialize thread pools, implement task submission and publishing mechanisms, set up result consumers, and orchestrate the overall computation flow.

Several pedagogical considerations motivated this choice. Both recursion and concurrency represent empirically challenging topics in computer science education. Lahtinen et al.'s survey of 530 students

¹ The programming exercise used in the experiment is provided with the dataset at <https://doi.org/10.5281/zenodo.16905779>.

ranked recursion as the third most challenging programming concept (Lahtinen et al., 2005), while Rashkovits and Lavy documented how students struggle profoundly with multi-threaded design, often ignoring parallelism requirements entirely (Rashkovits & Lavy, 2020).

Concurrency was selected as the more pedagogically appropriate choice for several reasons. First, concurrent programming represents a critical skill in modern software development, requiring students to understand complex concepts including thread coordination, race conditions, and synchronization mechanisms. Second, the multi-component architecture of concurrent programming tasks mirrors real-world software development challenges, where students must orchestrate multiple interconnected systems rather than implement isolated algorithms. Third, this domain tests both conceptual understanding and practical implementation skills, providing a comprehensive assessment of student learning.

The complexity of concurrent programming also created a research context where students would naturally seek assistance when encountering difficulties. This allowed us to observe how different support modalities affect learning in challenging domains where help-seeking behavior is common. The multi-file structure created realistic debugging and integration scenarios, enabling us to study how students navigate complex programming tasks with varying levels of AI support. The exercise was challenging enough to require sustained effort and problem-solving, yet achievable enough that motivated students could complete it within the time limit.

4.4. Data collection instruments

This study employed multiple complementary data sources to capture student performance, learning outcomes, and psychological experiences during the programming exercise. Two complementary measures comprehensively captured the learning outcomes: Knowledge Assessment (KA) for conceptual understanding and Code Comprehension (CC) for practical code recognition abilities. Table 1 presents all measures, their timing, scoring methods, and abbreviations used throughout this manuscript. Time points are denoted as T1 (pre-intervention) and T2 (post-intervention).

Performance data. The Artemis learning platform automatically captured EP and duration (ED) for all participants. Performance was measured as the percentage of test cases passed (0 %–100 %), while duration represented the elapsed time from exercise start to final submission in minutes. The platform also recorded comprehensive timestamps for each Git push, enabling analysis of submission patterns and attempt frequency in the future.

Surveys and assessments. Participants completed online surveys at two time points:

Pre-intervention (T1): Collected demographic information (gender, age, major), self-assessed programming experience (4-level ordinal scale), and the Knowledge Assessment (KA_{T1}), which included six objective knowledge questions. These were designed to assess students' understanding of fundamental concurrency concepts, including thread coordination, race conditions, and synchronization mechanisms, providing a baseline for measuring conceptual learning gains specific to the exercise content.

Post-intervention (T2): Included the identical Knowledge Assessment (KA_{T2}) to measure learning gains. Code Comprehension (CC) presented participants with a skeleton code implementation similar to the exercise, containing three major gaps. For each gap, students selected the correct code snippet from three options, testing their understanding of concurrent programming patterns. The survey also included psychological measures using validated scales (Intrinsic Motivation IM (Ryan, 1982), cognitive load subscales ICL/GCL/ECL, and Frustration FR). For AI-assisted groups only, eight perception items assessed their experience with the AI tool.

Table 1

Study measures and instruments across experimental timeline. Knowledge Assessment (KA) and Code Comprehension (CC) scores are calculated as the sum of correct answers. Likert-scale measures (IMI, cognitive load subscales, FR) are reported as mean scores across items. All measures were collected via online surveys except EP and duration, which were automatically recorded by the learning management system.

Measure	Items	Scoring	Abbreviation
<i>Baseline (T1)</i>			
Demographics (gender, age, major)	3	Categorical	—
Programming experience	1	Ordinal (4 levels)	—
Knowledge Assessment	6	Sum of correct (0–6)	KA _{T1}
<i>Intervention (During exercise)</i>			
Exercise performance	—	Percentage (0 %–100 %)	EP
Exercise duration	—	Minutes	ED
<i>Post-intervention (T2)</i>			
Knowledge Assessment	6	Sum of correct (0–6)	KA _{T2}
Code Comprehension	3	Sum of correct (0–3)	CC
Intrinsic Motivation (Interest & Enjoyment)	7	Mean of items (5-point Likert)	IM
Intrinsic Cognitive Load (task complexity)	2	Mean of items (5-point Likert)	ICL
Germane Cognitive Load (learning effort)	2	Mean of items (5-point Likert)	GCL
Extraneous Cognitive Load (inefficient processing)	3	Mean of items (5-point Likert)	ECL
Frustration level	1	Single item (5-point Likert)	FR
<i>AI groups only (T2)</i>			
Overall experience	1	Single item (5-point scale)	—
Ease of use	1	Single item (5-point Likert)	—
Comparison to traditional methods	1	Single item (5-point Likert)	—
Feedback helpfulness	1	Single item (5-point Likert)	—
Understanding queries	1	Single item (5-point Likert)	—
Help with exercises	1	Single item (5-point Likert)	—
Improved concept understanding	1	Single item (5-point Likert)	—
General helpfulness	1	Single item (5-point Likert)	—

Quality assurance. Surveys included attention-check questions (e.g., “Please select ‘Agree’”) to identify inattentive responses. Additionally, survey questions were tailored to experimental group. *IRIS* users received questions about Iris, *GPT* users about ChatGPT, ensuring contextual relevance while maintaining comparability across conditions.

Summary. Taken together, these measures form an educational data analytics setup: LMS log data (EP, ED), pre-post assessments (KA_{T1}, KA_{T2}, CC), and psychological scales (IM, ICL, GCL, ECL, FR) can be linked at the individual level to analyze how different support conditions affect students' performance, cognitive processing, and motivational states during an authentic course activity.

The next section will detail how we calculated the scores for this setup.

4.5. Calculation of composite scores

To facilitate comprehensive analysis, the study computed several composite measures from the collected data. These measures capture key learning outcomes and psychological constructs relevant to the research questions. The analysis calculated all composite scores using standardized procedures to ensure reliability and validity across the experimental conditions.

Knowledge assessment (KA) scores. The analysis calculated Knowledge Assessment scores by evaluating participants' responses to multiple-choice questions that test their understanding of concurrency programming concepts. The scoring procedure compared responses to correct answers, assigning 1 point for each correct response and 0 points for incorrect responses. The analysis summed the correct answers from the

six Knowledge Assessment multiple-choice questions to calculate participants' total Knowledge Assessment scores. The study applied this scoring procedure to both the pre-survey (KA_{T1}) and post-survey (KA_{T2}), with scores ranging from 0 to 6 points. The pre-survey assessment established baseline knowledge levels before the experimental intervention, while the post-survey assessment measured knowledge acquisition following the tutorial session. This approach enabled the calculation of learning gains by comparing pre- and post-intervention knowledge levels.

Code comprehension (CC). The study designed the Code Comprehension score to assess participants' ability to understand and interpret programming code beyond basic conceptual knowledge. The analysis calculated this measure by summing the correct responses (1 point each) from three fill-in-the-gap questions administered in the post-survey at T2. These questions required participants to analyze code snippets and identify missing elements, testing their practical understanding of programming constructs and logic flow. The resulting Code Comprehension scores ranged from 0 to 3 points, with higher scores indicating better ability to comprehend and work with programming code. This measure complemented the general knowledge assessment by focusing specifically on code interpretation skills, which are crucial for programming proficiency.

Intrinsic motivation inventory (IMI). The study measured intrinsic motivation to assess participants' engagement and enjoyment during the learning experience. The analysis evaluated this construct using seven items from the well-validated Interest & Enjoyment subscale of the Intrinsic Motivation Inventory (Ryan, 1982), with responses collected on a 5-point Likert scale ranging from 1 (strongly disagree) to 5 (strongly agree). To calculate the composite IM score, the analysis mapped responses to their corresponding numerical values and computed the mean across all seven items, with careful attention to reverse-coded items. The final IM score ranged from 1 to 5, with higher values indicating greater intrinsic motivation and enjoyment in the learning activity. Internal consistency for this scale reached high levels (Cronbach's $\alpha = .862$), confirming the reliability of the measure.

Cognitive load (CL) components. The study assessed cognitive load using the multidimensional instrument developed by Klepsch et al. (2017), which distinguishes between different types of mental effort required during learning, namely Intrinsic (ICL), Germane (GCL) and Extraneous Cognitive Load (ECL).

Intrinsic Cognitive Load (ICL) represents the mental effort inherent in the learning material itself and is the answer to the question: How much information must be understood and connected by the learner to comprehend the material. Germane Cognitive Load (GCL) reflects the mental effort devoted to processing and constructing knowledge schemas. This is the mental effort that leads to learning outcomes and represents productive cognitive effort. Extraneous Cognitive Load (ECL) refers to the mental effort caused by poor instructional design or irrelevant information; thus, it is the load that hinders meaningful learning and should be minimized.

All cognitive load items used a 5-point Likert scale ranging from 1 (very low) to 5 (very high). To validate the measurement instrument, we conducted a Principal Component Analysis (PCA) on all 7 items ($N = 452$). The analysis confirmed excellent sampling adequacy ($KMO = 0.805$) and revealed a two-factor structure with the first component explaining 48.7 percent of variance and the second component explaining 16.5 percent of variance. All items demonstrated meaningful loadings on the first principal component (range: 0.275–0.422), with GCL and ECL items showing the strongest contributions. The full 7-item scale achieved excellent internal consistency (Cronbach's $\alpha = 0.82$).

The ICL and GCL subscales each contain only 2 items, which severely limits traditional reliability assessment. Cronbach's α is not reported for these subscales as two-item scales frequently yield

unreliable or nonsensical reliability estimates. While the PCA confirms that all items contribute meaningfully to a unified cognitive load construct, this does not establish reliability for individual subscales. Therefore, interpretations of ICL and GCL subscale differences should be treated as exploratory findings requiring replication with more comprehensive, validated instruments. ICL was calculated as the average of items 1–2, GCL as the average of items 3–4, and ECL as the average of items 5–7 (Cronbach's $\alpha = 0.737$ for the 3-item ECL subscale). The primary analysis utilized an overall Cognitive Load index (CL) as the mean across all 7 items, providing a robust and reliable measure of total mental effort.

Exercise duration (ED). The study obtained exercise performance metrics directly from the Artemis learning platform. One important measure was exercise duration, which the analysis calculated by determining the time elapsed between when students started the programming exercise and the last submission. This approach ensures precise measurement of actual time spent on the programming exercise, eliminating potential biases from self-reported time estimates.

4.6. Experimental procedure

Before the experiment, all tutors participated in a structured preparatory training designed to ensure consistent execution across all tutorial groups (see Fig. 1). They first received a thorough theoretical introduction to the experiment, covering its objectives, underlying rationale, and planned procedure. This introduction ensured that tutors understood the purpose of the study and the sequence of activities. The training then continued with a complete dry run of the entire procedure, enabling tutors to experience the experiment from a participant's perspective and to develop a precise understanding of each step. The final analysis excluded the data collected during this preparatory session. Tutors received a detailed written protocol specifying the sequence of activities, timing, and instructions to be given to participants. They could consult an experienced researcher at any time during preparation and execution to resolve uncertainties or address unforeseen issues. This arrangement ensured that tutors could conduct the experiment independently within their sessions while maintaining procedural fidelity across groups.

The announcement stated that students would receive course bonus points for completing the experiment, regardless of the answers they provided as long as they participated seriously. Completion required finishing the pre- and post-surveys and the programming exercise in a logical sequence within their assigned tutorial group. The bonus points did not depend on the content of the survey responses or on the score students achieved in the programming exercise.

Each tutorial group session started with a brief summary of the previous lecture and a walkthrough of the homework assignment. This served to reactivate prior knowledge and establish a productive working atmosphere. Before the experiment, the tutor explained to students the study's general framework. Participation was voluntary and could be discontinued at any time without penalty. Before participation, the researchers informed students about purpose, procedures, data handling, and their rights under GDPR (access, rectification, deletion). The tutor also informed students that they would complete two surveys, one before and one after the programming exercise, to assess their understanding and experiences. Fig. 2 depicts the tutorial group process.

Following this introduction, the tutor presented the experiment's specific objectives and outlined the upcoming procedure. Before starting the exercise, all students individually completed the pre-survey assessing their initial understanding of concurrency concepts and some demographic data. Tutors emphasized that it was critical for students to complete the survey independently without external assistance and supervised the process to ensure compliance. To control for any violations of these rules, students were able to opt-out of their data being used at the end of the study through the last question in the post-survey, where

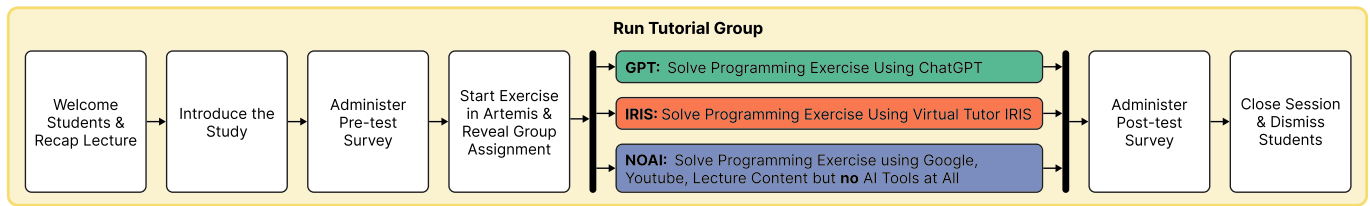


Fig. 2. Overview of the experimental procedure within tutorial session groups. Each 90-minute session began with introduction and consent procedures, followed by the pre-survey and knowledge assessment. Students then received their randomized group assignment (ChatGPT, Iris, or No-AI) and completed the concurrent programming exercise. The session concluded with the post-survey containing knowledge assessment, cognitive load measures, and experience ratings. Tutors supervised all sessions to ensure protocol adherence.

they could indicate if they had received external help and therefore their data should not be used. This opt-out option did not affect their bonus points and carried no disadvantages.

Once all students had completed the pre-survey, they proceeded to the programming exercise within *Artemis*, which they completed individually under one of the three assigned conditions described in Section 4.2. All students had 90 minutes to complete the exercise and could submit their solutions at any time during this period. Tutors ensured that students did not receive external help and did not discuss the exercise or the study with peers.

After submitting their code, students completed a post-survey to assess their learning outcomes and experiences during the experiment. The same non-interference rules applied as during the pre-survey and programming phase. Throughout the session, tutors clarified and enforced the rules to ensure consistency and fairness. Tutors and fellow students did not provide assistance concerning the exercise content. The study prohibited students from discussing the exercise, surveys, or any details about the study until they completed all components, and they could not share information about the study with peers scheduled for future sessions. This approach maintained uniform experimental conditions, protected data accuracy, and ensured fairness across all tutorial group sessions.

Tutors conducted the experiment independently within their sessions. Experienced researchers remained available for immediate consultation in case of questions. This structure supported an effective integration of the experiment into the course, facilitating participation and consistent data collection.

5. Data analysis

The data analysis consisted of merging and pseudonymizing datasets, data cleaning and validation, and conducting statistical tests. The analysis employed R as the primary statistical software.

5.1. Data cleaning and quality assurance

Data preparation began with a pseudonymized dataset that integrated multiple sources: pre-surveys, post-surveys, *Artemis* platform data such as exercise performance and duration, and chat interaction logs for AI conditions. To protect participant privacy, the researchers replaced all personal identifiers with pseudonymized codes prior to analysis and stored the data on secure university servers; only the merging researcher had access to identifiable data. The research team worked exclusively with this pseudonymized dataset, ensuring that no identifying information was accessible during the analytical phase.

The data preprocessing methodology implemented quality control measures to ensure analytical validity and dataset integrity. Initial preprocessing operations focused on eliminating extraneous elements, including the systematic removal of empty columns, metadata irrelevant to the research objectives (such as internal session tokens), and redundant variables generated during data merging procedures.

In order to maintain data integrity, the study applied exclusion criteria systematically in a specific sequence to ensure cascading effects

Table 2

Participant exclusions by category (initial $N = 452$). The table shows the systematic application of data quality filters, ordered by number of exclusions. Timing inconsistencies were the largest category, including violations of logical temporal sequences. The final analytical sample of 275 participants (60.8 % retention) represents those with complete, valid data across all measures. The 39.2 % exclusion rate, while substantial, ensures data integrity for a large-scale field experiment conducted across 35 tutorial sessions with 19 different tutors.

Exclusion reason	Number of participants	Percent (%)
Timing inconsistencies	48	10.6
Outside study window	40	8.8
Attention checks failed	26	5.7
Chat history presence	22	4.9
Demographic data issues	20	4.4
Implausible or missing exercise duration	17	3.8
Opt-out	4	0.9
Total excluded	177	39.2

were properly captured. The analysis anchored the investigation to a predefined study timeframe. From the initial 452 participants, the study applied the following filters in order: (1) temporal coherence checks excluding participants who completed the pre-survey after starting the programming exercise, showed less than 10 minutes or more than 2 hours between pre- and post-surveys, or had submission timestamps that violated logical sequencing; (2) attention check validation, excluding participants who failed embedded questions (e.g., “Please select ‘Agree’”); (3) removing entries initiated before the start of the timeframe; (4) submission validation, excluding participants with empty exercise submission histories or no valid submissions before their post-survey; (5) self-opt-out filtering; (6) condition-specific requirements, where AI groups required evidence of chat interaction history; (7) demographic plausibility filters, excluding participants with ages more than three standard deviations from the sample mean; and (8) exercise duration filters. These quality control measures resulted in the exclusion of 177 participants (39.2 % of initial sample), yielding a final sample of 275 students with complete and valid data. The exclusions were distributed as shown in Table 2.

All exclusion criteria were applied uniformly across all experimental conditions to ensure unbiased data cleaning. Because excluded participants did not complete the full experimental protocol, it's not possible to compare their characteristics on the primary outcome measures with those who were included in the analysis. However, the analysis checked whether the exclusion pattern differed systematically between experimental groups, which would suggest differential attrition that could bias results.

The exclusion rate was substantial, but not unexpected for a large-scale field experiment conducted across 35 tutorial sessions with 19 different tutors, where maintaining strict scientific protocol adherence presents inherent challenges. The systematic application of predefined quality criteria ensures the integrity of findings while maintaining sufficient statistical power across all three experimental conditions.

Table 3

Participant characteristics and randomization checks ($N = 275$). The table presents demographic distributions and statistical tests confirming successful randomization across experimental groups. Chi-square (C-S) tests for categorical variables and ANOVA (AN) for continuous variables or Kruskal-Wallis (K-W) tests for ordinal variables showed no significant differences between groups (all $p > .05$), indicating equivalent baseline characteristics. Gender distribution was balanced (approximately 50 % female), programming experience was predominantly at the beginner and intermediate levels, and age averaged 23 years across all conditions.

Characteristic	Test results			Descriptive statistics		
	Test	Statistic	p -value	GPT ($n = 88$)	IRIS ($n = 91$)	NOAI ($n = 96$)
Gender	C-S	$\chi^2 = 1.37$	.504			
Female				49 (55.7 %)	44 (48.4 %)	46 (47.9 %)
Male				39 (44.3 %)	47 (51.6 %)	50 (52.1 %)
Programming Experience	K-W	$\chi^2 = 0.80$	.669			
None				13 (14.8 %)	17 (18.7 %)	17 (17.7 %)
Beginner				41 (46.6 %)	39 (42.9 %)	47 (49.0 %)
Intermediate				32 (36.4 %)	35 (38.5 %)	32 (33.3 %)
Expert				2 (2.3 %)	0 (0.0 %)	0 (0.0 %)
Age (years)	AN	$F = 1.77$	.173	22.53 (2.64)	23.07 (2.92)	23.29 (2.80)

C-S = Chi-square test; K-W = Kruskal-Wallis test; AN = Analysis of Variance. Descriptive statistics are count and percentage for categorical variables and mean and standard deviation for continuous variables.

5.2. Statistical analysis

The study conducted a comprehensive series of statistical analyses to examine group differences and relationships between variables.²

The primary analytical framework comprised four main statistical approaches, each selected for specific aspects of the educational intervention. The analysis used one-way ANOVA to compare continuous outcomes (e.g., exercise performance, cognitive load, motivation) across the three experimental groups. When Levene's test (Levene, 1960) indicated heterogeneous variances the analysis employed Welch's ANOVA (Delacre et al., 2019; Welch, 1951) as an alternative ($p < 0.05$). This adaptation was crucial because exercise performance, in particular, showed different spreads across groups, with AI-assisted students exhibiting wider performance ranges. Effect sizes were reported as generalized eta-squared (η_p^2), providing a standardized measure of practical significance essential for educational interventions (Lakens, 2013; Richardson, 2011).

The analysis employed a repeated measures ANOVA to explore the fixed effects and interactions of the experimental groups on the difference between knowledge before and after the intervention, using KA_{T1} and KA_{T2} data while accounting for the within-subject nature of pre-post measurements.

For categorical demographic variables, the analysis used Chi-square tests to assess randomization success by testing independence between group assignment and gender distribution. This was essential to verify that the randomization process had not inadvertently created group imbalances that could confound treatment effects. Programming experience, being ordinal rather than continuous, required the Kruskal-Wallis test (Kruskal & Wallis, 1952), a non-parametric alternative that respects the ordered nature of experience levels (None, Beginner, Intermediate, Expert) without assuming equal intervals between categories.

The analysis selected Tukey HSD (Tukey, 1953) and Games-Howell (Games & Howell, 1976) as post-hoc tests to explore the differences between the distinct groups. This selection followed a systematic decision tree tailored to the characteristics of educational data. Given the exploratory nature of comparing novel AI tools, the analysis prioritized appropriate error control while maintaining sufficient power to detect educationally meaningful differences. Tukey HSD was employed when

assumptions were met (homogeneous variances, balanced design) as it optimally controls familywise error rate across all pairwise comparisons, which is critical when examining three treatment conditions simultaneously. Games-Howell was selected as an alternative for heterogeneous variances, particularly important for variables like FR and EP where AI tools might create differential variability.

To compare ChatGPT and Iris users on AI-specific perception measures, the analysis used independent samples t-tests. Welch's correction (Welch, 1951) was applied when variance equality was violated, accounting for potential differences in response variability between tools with different interaction paradigms. Throughout all analyses, standardized effect sizes (generalized eta-squared for ANOVAs (Richardson, 2011), Cohen's d for t-tests (Cohen, 1988)) complemented significance testing.

6. Results

This section presents the empirical findings from the randomized controlled trial examining the effects of different AI support modalities on programming education outcomes. The presentation first establishes the equivalence of the experimental groups through demographic and baseline comparisons, then examines primary learning outcomes, followed by analyses of affective measures and cognitive load. Finally, the analysis presents direct comparisons between the two AI groups to understand their relative strengths and limitations.

6.1. Randomization checks and group equivalence

The analysis tested gender distribution, programming experience levels, and age concerning their distribution in the different experimental groups to verify that the randomization was successful. Table 3 summarizes the results.

Chi-square tests for categorical variables and ANOVA for continuous variables confirmed successful randomization. Gender distribution was balanced across groups, with approximately half of participants being female in each condition. Programming experience levels, ranging from none to expert, showed similar distributions across groups as assessed by Kruskal-Wallis test. Age was comparable across conditions, with participants primarily in their early twenties as expected for a university sample. These analyses confirm that random assignment produced equivalent groups at baseline, supporting valid between-group comparisons.

² The code for the analysis and the dataset are available at <https://doi.org/10.5281/zenodo.16905779>.

Table 4

Primary learning outcomes: ANOVA results and descriptive statistics ($N = 275$). The table presents one-way ANOVA results for EP and CC, showing significant group differences for EP ($p < .001$, $\eta_g^2 = .179$) but not for CC ($p = .136$). EP demonstrates the immediate performance advantage of AI assistance, with *GPT* users achieving 2.4× higher scores than *NOAI*. The non-significant difference in CC suggests that enhanced EP did not translate to improved CC abilities. W indicates Welch's ANOVA was used due to heterogeneous variances.

Measure	Test results				Means and standard deviations		
	Test	Statistic (F)	Effect Size (η_g^2)	p -value	<i>GPT</i> ($n = 88$)	<i>IRIS</i> ($n = 91$)	<i>NOAI</i> ($n = 96$)
Exercise Performance (%)	AN	29.69	.179	< .001	71.84 (39.65)	57.50 (37.36)	29.85 (36.17)
Exercise Duration (min)	W-AN	1.97	.022	.142	88.16 (12.22)	90.93 (8.69)	91.28 (10.55)
Code Comprehension	AN	2.01	.015	.136	1.36 (0.91)	1.43 (0.87)	1.18 (0.89)

AN = Analysis of Variance; W-AN = Welch's ANOVA.

Table 5

Post-hoc pairwise comparisons for exercise scores. Tukey HSD test results show that all three groups differed significantly from each other. *GPT* users outperformed both *IRIS* ($p = .031$) and *NOAI* ($p < .001$) groups, while *IRIS* users also significantly outperformed the *NOAI* group ($p < .001$). Effect sizes (Cohen's d) indicate large practical differences, with the *GPT* vs. *NOAI* comparison showing the strongest effect ($d = 1.12$). These results demonstrate a performance hierarchy: *GPT* > *IRIS* > *NOAI*.

Comparison	Mean difference	Effect size (d)	p -value
<i>GPT</i> vs. <i>IRIS</i>	14.339	0.38	.031
<i>GPT</i> vs. <i>NOAI</i>	41.988	1.10	< .001
<i>IRIS</i> vs. <i>NOAI</i>	27.649	0.76	< .001

All comparisons use the Tukey HSD post-hoc test.

Regarding attrition patterns, the final sample distribution shows balanced retention across experimental conditions: *GPT* ($n = 88$ of 154, 57.1 %), *IRIS* ($n = 91$ of 150, 60.7 %), and *NOAI* ($n = 96$ of 148, 64.9 %). The similar group sizes suggest that participant exclusions were distributed approximately equally across conditions, providing no evidence of differential attrition by experimental group. This pattern indicates that the 39.2 % attrition rate was likely random rather than systematic, reducing concerns that particular treatment conditions selectively lost participants with specific characteristics.

6.2. Primary learning outcomes

Table 4 presents the ANOVA results for primary learning outcomes across experimental conditions.

Exercise performance showed substantial group differences, with post-hoc comparisons in Table 5 revealing that both AI-assisted groups (*IRIS* and *GPT*) achieved higher scores than the *NOAI* group, and *GPT* users scored higher than *IRIS* users, meaning they could solve more sub-tasks of the programming exercise in the tutorial session. Despite these performance differences, exercise duration remained similar across all three groups, suggesting that AI assistance improved efficiency rather than simply allowing more time on task.

Table 6 presents the repeated measures ANOVA for knowledge scores before and after the intervention (KA_{T1} and KA_{T2}), which revealed a significant main effect of time, indicating that learning occurred across all experimental groups during the intervention. However, neither the main effect of group nor the interaction between time and group reached significance, indicating no significant differences in learning gains between support conditions. Similarly, as shown in Table 4, Code Comprehension, measuring students' post-intervention ability to understand the code, showed no significant differences between groups.

Fig. 3 visualizes these learning outcomes through three panels. The KA_{T1} and KA_{T2} scores boxplot illustrates the parallel increases from T1 to T2 across groups. The Code Comprehension performance boxplot shows the similar distributions of scores across conditions. Most notably, the Exercise Performance violin plot reveals distinct distribution patterns: *GPT* users concentrated at the upper end of the scale, *NOAI* users clustered at lower scores, while *IRIS* users displayed a broad distribution spanning the entire range.

6.3. Affective measures and cognitive load

Tables 7 and 8 present results for affective factors and cognitive load measures. Frustration levels differed significantly across groups, with both AI groups reporting lower frustration than the *NOAI* group, while the two AI groups did not differ from each other ($p = .886$). Intrinsic motivation showed a different pattern: only the *IRIS* group demonstrated significantly higher motivation compared to the *NOAI* group ($p < .001$), while the *GPT* group showed no motivational advantage over traditional resources ($p = .076$).

Cognitive load analysis revealed differentiated patterns across components. ICL showed no significant differences between groups ($p = .080$), with all three conditions reporting similar task complexity. For GCL, both AI groups showed significant reductions compared to *NOAI* ($p < .001$). The *GPT* group demonstrated the largest reduction ($d = -0.77$), followed by the *IRIS* group ($d = -0.56$), though the difference between AI groups was not significant ($p = .191$). Similarly, ECL decreased significantly in both AI groups compared to *NOAI* ($p < .001$). The *GPT* group showed the strongest reduction ($d = -0.87$) and the *IRIS* group

Table 6

KA_{T1} and KA_{T2} scores: Repeated measures ANOVA results ($N = 275$). The analysis reveals a significant main effect of time ($p < .001$, $\eta_g^2 = .042$), indicating that learning occurred from T1 to T2 across all conditions. However, neither the group main effect ($p = .311$) nor the time $\times$ group interaction ($p = .773$) reached significance, demonstrating that while all students learned from the exercise, AI assistance did not differentially affect conceptual knowledge gains.

Source	Main effects and interaction			Descriptive statistics			
	F	η_g^2	p -value	Time	<i>GPT</i> ($n = 88$)	<i>IRIS</i> ($n = 91$)	<i>NOAI</i> ($n = 96$)
Time	88.23	.042	< .001	T1	2.33 (1.82)	2.18 (1.95)	1.92 (1.86)
Group	1.17	.007	.311	T2	3.16 (1.90)	2.89 (1.92)	2.77 (1.98)
Group $\times$ Time	0.26	< .001	.773	Gain	0.83	0.71	0.85

Descriptive statistics are means and standard deviations in parentheses. Gain scores represent the mean difference between T1 and T2.

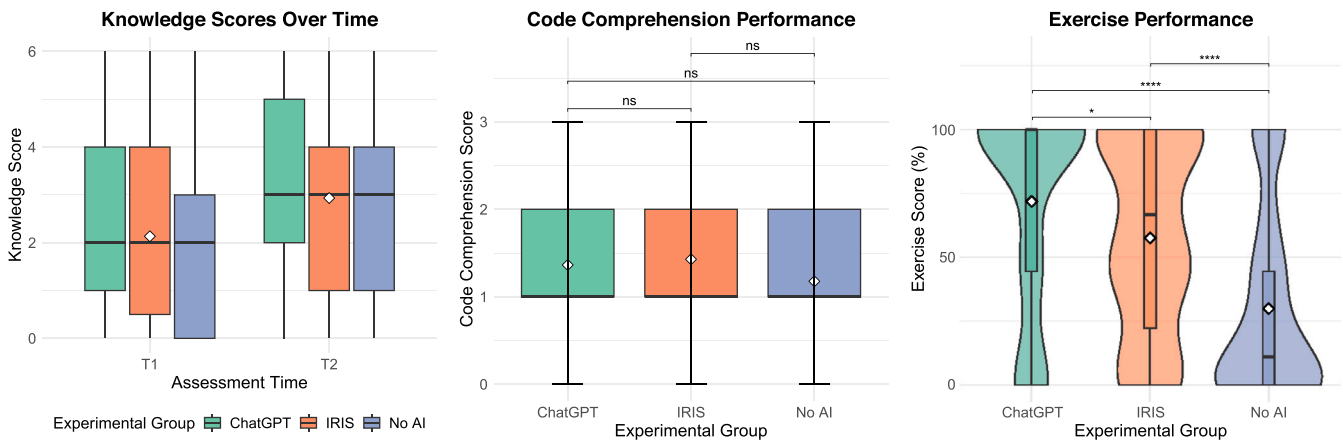


Fig. 3. Learning and performance outcomes across experimental conditions. (1) Knowledge scores over time showing parallel increases from T1 to T2 across all groups, with no differential learning gains despite AI assistance. (2) Code Comprehension scores showing no significant differences between groups the ability to understand the code. (3) Exercise Performance scores (violin plot with embedded boxplot) revealing the dramatic performance advantage of AI-assisted groups, particularly *GPT*, while *IRIS* users showed a broad distribution across the possible range of scores.

Table 7

Affective measures and cognitive load: ANOVA results and descriptive statistics. FR shows significant reduction in both AI groups. IM reveals a unique advantage for the *IRIS* group. Cognitive load analysis shows that intrinsic load remains unchanged, while germane and extraneous loads are significantly reduced in both AI groups.

Measure	Test results				Means and standard deviations		
	Test	Statistic (<i>F</i>)	Effect size (η_g^2)	<i>p</i> -value	<i>GPT</i> (n = 88)	<i>IRIS</i> (n = 91)	<i>NOAI</i> (n = 96)
<i>Affective Factors</i>							
Frustration	W-AN	23.41	.054	< .001	3.13 (1.21)	3.21 (1.18)	4.09 (1.00)
Intrinsic Motivation	AN	7.54	.053	< .001	2.64 (0.65)	2.82 (0.70)	2.42 (0.74)
<i>Cognitive Load</i>							
Intrinsic	AN	2.55	.018	.080	3.81 (0.74)	3.59 (0.84)	3.81 (0.76)
Germane	W-AN	16.08	.026	< .001	3.56 (1.12)	3.83 (0.91)	4.28 (0.69)
Extraneous	AN	19.24	.124	< .001	3.13 (0.92)	3.31 (0.85)	3.86 (0.74)

AN = Analysis of Variance; W-AN = Welch's ANOVA.

Table 8

Affective measures and cognitive load: Post-hoc pairwise comparisons. Games-Howell tests were used for variables with heterogeneous variances, while Tukey HSD was used for homogeneous variances. Effect sizes (Cohen's *d*) indicate medium to large practical significance for most comparisons, with FR and extraneous load showing the strongest effects.

Measure	Test	Mean difference	Effect size (<i>d</i>)	<i>p</i> -value
<i>FR</i>				
<i>GPT</i> vs. <i>IRIS</i>	G-H	−0.084	−0.07	.886
<i>GPT</i> vs. <i>NOAI</i>	G-H	−0.969	−0.87	< .001
<i>IRIS</i> vs. <i>NOAI</i>	G-H	−0.885	−0.81	< .001
<i>IM</i>				
<i>GPT</i> vs. <i>IRIS</i>	TK	−0.170	−0.25	.234
<i>GPT</i> vs. <i>NOAI</i>	TK	0.225	0.32	.076
<i>IRIS</i> vs. <i>NOAI</i>	TK	0.395	0.55	< .001
<i>GCL</i>				
<i>GPT</i> vs. <i>IRIS</i>	G-H	−0.267	−0.26	.191
<i>GPT</i> vs. <i>NOAI</i>	G-H	−0.719	−0.77	< .001
<i>IRIS</i> vs. <i>NOAI</i>	G-H	−0.452	−0.56	< .001
<i>ECL</i>				
<i>GPT</i> vs. <i>IRIS</i>	TK	−0.175	−0.20	.342
<i>GPT</i> vs. <i>NOAI</i>	TK	−0.729	−0.87	< .001
<i>IRIS</i> vs. <i>NOAI</i>	TK	−0.553	−0.70	< .001

G-H = Games-Howell; TK = Tukey HSD.

also demonstrated substantial reduction ($d = -0.70$), with no significant difference between AI groups ($p = .342$).

The motivational patterns complemented the cognitive load findings. While both AI tools successfully reduced frustration to similar degrees (both $p < .001$ vs. *NOAI*; $p = .886$ between AI groups), only the *IRIS* group

produced a significant increase in IM compared to *NOAI* ($p < .001$, $d = 0.55$). *IRIS* users reported higher intrinsic motivation than *NOAI*, while *GPT* showed no significant motivational benefit over *NOAI* ($p = .076$).

Fig. 4 illustrates the FR and IM findings through boxplots, showing the parallel frustration reduction in both AI groups alongside the *IRIS* group's unique motivational advantage. Fig. 5 presents the three cognitive load components through side-by-side boxplots, demonstrating how both AI tools reduced germane and extraneous load while leaving intrinsic load unchanged, with *GPT* showing descriptively larger reductions in both learning-related load components.

6.4. Direct comparison of AI tools

Table 9 provides head-to-head comparisons of students' perceived experiences with the two AI tools. The analysis revealed several significant differences in user perceptions that favor ChatGPT. Students perceived ChatGPT as significantly easier to use compared to *Iris* ($p = .003$). *GPT* users rated its feedback as more helpful than *IRIS* users ($p = .004$). *GPT* users also felt the tool better understood their queries ($p = .011$) and was more effective at helping resolve exercise-related issues ($p < .001$). Additionally, *GPT* users reported greater perceived improvement in their concept understanding compared to *IRIS* users ($p = .001$).

However, not all perceptual comparisons favored ChatGPT. Differences in overall perceived AI experience favoring *Iris* approached but did not reach significance, and both tools were rated similarly when students compared them to traditional learning methods. Perceived general helpfulness for understanding showed no significant difference between the AI tools, suggesting students found both tools subjectively valuable despite their different approaches.

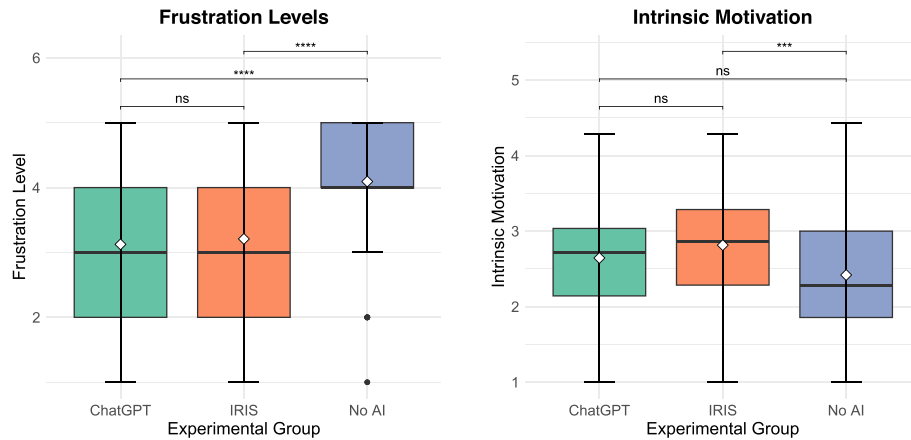


Fig. 4. Affective measures across experimental conditions comparing the *NOAI*, *GPT*, and *IRIS* groups. Left panel shows FR levels measured on a 5-point scale, with both AI groups demonstrating significantly lower FR than the *NOAI* group ($p < .001$). Right panel displays IM scores from the Interest and Enjoyment subscale of the Intrinsic Motivation Inventory, showing a unique motivational advantage for the *IRIS* group compared to the *NOAI* group ($p < .001$), while the *GPT* group showed no significant motivational benefit. Boxplots display median values, interquartile ranges, and individual outliers.

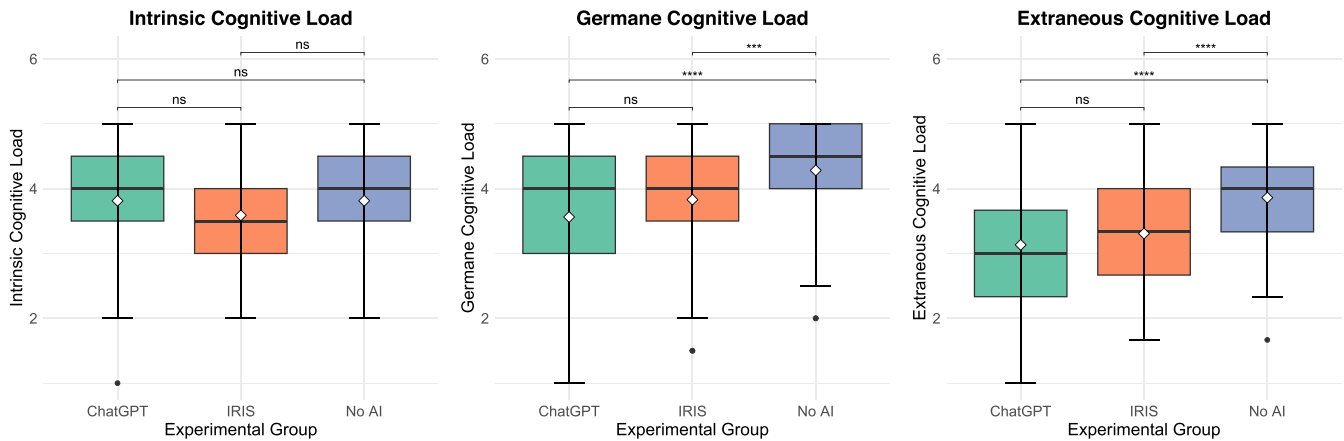


Fig. 5. Cognitive load measures across experimental conditions comparing the *NOAI*, *GPT*, and *IRIS* groups. Left panel shows ICL (task complexity), which did not differ significantly between groups ($p = .080$). Middle panel displays GCL (learning-oriented effort), with both AI groups showing significantly lower values than the *NOAI* group ($p < .001$). Right panel presents ECL (inefficient processing), with both AI groups significantly reducing values compared to traditional resources ($p < .001$). Boxplots display median values, interquartile ranges, and individual outliers.

Table 9

Direct comparison of AI tool experiences between the *GPT* and *IRIS* groups ($n = 179$). The table presents results from independent samples *t*-tests comparing user perceptions and experiences with the two AI assistants across multiple dimensions. All measures used 5-point Likert scales (1 = strongly disagree, 5 = strongly agree). Statistical tests employed Welch's *t*-test to account for unequal variances. Effect sizes are reported as Cohen's *d*, with positive values favoring ChatGPT.

Measure	Test results			Means and standard deviations	
	Statistic (<i>t</i>)	Cohen's <i>d</i>	<i>p</i> -value	ChatGPT ($n = 88$)	Iris ($n = 91$)
<i>General AI Perceptions</i>					
Overall AI experience	-1.90	-0.29	.059	2.10 (1.04)	2.40 (1.02)
AI was easy to use	3.00	0.45	.003	4.36 (0.82)	3.97 (0.95)
More effective than traditional	1.47	0.22	.143	3.85 (1.15)	3.60 (1.10)
<i>AI Helpfulness & Impact</i>					
Feedback was helpful	2.96	0.44	.004	3.98 (0.96)	3.54 (1.03)
AI understood queries	2.57	0.38	.011	3.90 (0.86)	3.56 (0.90)
Helped resolve exercise issues	4.25	0.63	< .001	4.08 (0.83)	3.49 (1.00)
Improved concept understanding	3.30	0.49	.001	3.86 (1.05)	3.37 (0.93)
General helpfulness	0.94	0.14	.351	3.75 (1.14)	3.60 (0.93)

All tests are independent samples *t*-tests with Welch's correction for unequal variances.

7. Findings

The analysis reveals four key findings that illuminate how different AI support modalities influence programming education. These findings highlight important tensions between performance enhancement and learning effectiveness. Collectively, they suggest that one interpretation is that AI tools may function to help students complete challenging programming tasks rather than act as direct learning enhancers, with the critical distinction lying in how they provide this support.

Finding 1: AI as a Performance, Not Learning, Enhancer

Both AI tools significantly improved exercise performance without enhancing actual learning outcomes. All groups showed no significant differences in knowledge gains or code comprehension abilities despite vast differences in task completion rates, suggesting that failing or succeeding in the exercises might not be the decisive factor for learning.

Both AI-assisted groups (*IRIS* and *GPT*) achieved substantially higher exercise performance than the *NOAI* group, with *GPT* users reaching 71.84 % compared to 29.85 % for *NOAI* users, yet this performance advantage did not translate into superior learning. All three groups showed no significant differences in improvements in conceptual knowledge and code comprehension abilities. The constant exercise duration across groups indicates that students didn't spend more time with AI assistance; they simply accomplished more within the same timeframe. Both AI groups also reported significantly reduced frustration compared to the *NOAI* condition. This stark dissociation between performance enhancement and learning outcomes suggests that AI tools while improving task performance do not improve learning outcomes, but also don't hinder them.

Finding 2: Iris Balances Scaffolding and Cognitive Challenge

Exercise performance distributions revealed distinct patterns: *GPT* users clustered at high scores, *NOAI* users at low scores, while *IRIS* users were distributed across the full range. This preservation of performance variation, combined with germane cognitive load patterns, suggests that *Iris* achieves a unique balance between support and challenge.

GPT users clustered at high performance levels, creating a ceiling effect where nearly all students achieved similar high scores. The *NOAI* group showed the opposite pattern, clustering at the lower end of the performance spectrum. *IRIS*, however, produced a markedly different distribution: students spread across the entire performance range, preserving individual differences in achievement. The cognitive load data revealed a parallel pattern. While both AI tools significantly reduced GCL compared to the *NOAI* condition, the *GPT* group's reduction was descriptively larger than the *IRIS* group's. Though the direct comparison between AI tools did not reach statistical significance, the combination of performance distributions and cognitive load patterns suggests different mechanisms of support between the two AI systems.

Finding 3: Iris Improves Intrinsic Motivation

Only *Iris* provided students with a motivating challenge, achieving what *ChatGPT* could not: simultaneously reducing frustration while enhancing engagement. These results suggest that the interaction design of a digital tutor matters and can be the difference between a motivating challenge, an easy task, and a frustrating experience.

Both AI-assisted groups successfully reduced student frustration to similar degrees compared to the *NOAI* group. However, only *IRIS* translated this into increased intrinsic motivation, as *GPT* users showed no motivational benefits despite their higher exercise performance. This divergence occurred even though both tools enabled similar reductions in frustration and improvements in exercise completion. *Iris*'s specific design features may explain this pattern: its tutor-like interaction style enables students to overcome problems themselves by scaffolding its help, while keeping the problem's solution out of reach. In contrast, *ChatGPT*'s approach of providing direct solutions when requested appeared to leave motivation unchanged from baseline levels. The data suggest that the interaction design and pedagogical approach of an AI tutor influence motivational outcomes independently of its effects on performance or frustration.

Finding 4: ChatGPT is a Comfort Trap

Students perceived *ChatGPT* as significantly more helpful and easier to use, yet these preferences aligned with greater reductions in learning-related cognitive processing and did not lead to better learning outcomes. This perception-reality gap reveals how student preferences may mislead pedagogical decisions.

Students consistently rated *ChatGPT* more favorably than *Iris* across multiple measures, including ease of use, feedback quality, query understanding, and perceived learning support. These preferences aligned with the *GPT* group's greater reduction in GCL. Meanwhile, the *IRIS* group, which experienced enhanced intrinsic motivation and maintained higher cognitive engagement, received lower ratings on perceived helpfulness. This inverse relationship between student preferences and pedagogical outcomes aligns with existing literature showing that students' perceptions of learning can be inversely related to actual learning outcomes (Deslauriers et al., 2019), that satisfaction ratings have near-zero correlation with objective learning measures (Uttl et al., 2017), and that learners often prefer easier but less effective methods (Yan et al., 2016).

8. Discussion

This study reveals an intriguing relationship in AI-supported programming education: tools that significantly improve exercise performance do not automatically enhance conceptual learning. These findings must be interpreted within the constraints of the study's learning measures. Nevertheless, they indicate an interesting relationship between student success and understanding, while illuminating critical design considerations for educational AI systems.

In the following sections, this discussion examines the mechanisms underlying the performance-learning dissociation, explores how different scaffolding approaches affect student engagement and cognitive load, analyzes the motivational consequences of AI assistance design, examines the perception-reality gap in student preferences, and discusses the implications for educational practice and future research.

8.1. The performance-learning dissociation

The complete dissociation between exercise performance and learning outcomes (Finding 1) raises important questions about AI's role in education. One interpretation of this pattern is that AI may function in programming education by helping students complete constant-difficulty tasks across the knowledge spectrum rather than enhancing learning directly, although the learning measures employed were limited.

One lens for understanding these results is that traditional programming exercises of constant difficulty create a binary outcome: students with sufficient background knowledge succeed while others struggle

fruitlessly. AI assistance may break this pattern by providing support that partially compensates for knowledge deficits, potentially allowing more students to engage with the material. The constant exercise duration across groups reinforces this interpretation: students didn't spend more time with AI assistance; they simply accomplished more within the same timeframe, potentially helping students engage with learning opportunities that might otherwise be out of reach. This interpretation, that AI tools may help enable task completion rather than enhance learning directly, has implications for how educators might integrate AI into educational settings.

The dissociation also raises critical questions about assessment validity in AI-enabled environments. If students can achieve high scores on programming exercises without commensurate gains in understanding, traditional performance-based assessments may no longer serve as reliable indicators of learning. This echoes concerns raised by Denny et al. (2024) about the need to fundamentally reconceptualize assessment in the age of AI.

8.2. Scaffolding and the preservation of productive challenge

The distribution patterns of exercise performance reveal differences in how AI tools support student performance on challenging tasks (Finding 2). ChatGPT appears to have “lifted” nearly all students to high performance levels, essentially removing the challenge entirely and enabling uniform completion of difficult tasks. In contrast, the NOAI group showed limited success, with the constant-difficulty task proving challenging for most regardless of effort. Iris's pattern suggests something different: calibrated assistance. This approach offers enough help for students to approach tasks without revealing solutions. It preserves individual differences while potentially enabling broader engagement.

This preservation of performance variation suggests that Iris's scaffolded approach maintains what Vygotsky termed the “zone of proximal development,” the space where learning is both challenging and achievable (Vygotsky, 1978). The calibrated approach allows students to receive sufficient support to engage with constant-difficulty material without eliminating the productive struggle that drives understanding.

The cognitive load data provide a mechanistic explanation for these patterns. While both AI tools reduced GCL and ECL compared to the NOAI group, ChatGPT's reduction was descriptively more pronounced. Following Sweller's cognitive load framework (Sweller et al., 1998), germane load represents the mental effort devoted to schema construction and automation, the very processes that drive learning. Lower germane processing means that students did not invest as much active cognitive effort (Klepsch et al., 2017; Klepsch & Seufert, 2021), which suggests either improved *efficiency* (support reduces the amount of learning-oriented processing needed) or *dampened investment* (support reduces engagement in learning-relevant processing).

This finding aligns with prior research on intelligent tutoring systems showing that moderate scaffolding outperforms both minimal and maximal support (Jennings & Muldner, 2021). Iris appears to have found this “sweet spot,” providing enough assistance to prevent frustration while preserving sufficient challenge to maintain cognitive engagement. The tool's design, offering hints and guidance without revealing solutions, exemplifies the scaffolding functions Wood et al. (1976) identified: it recruits student engagement, reduces task complexity to manageable levels, maintains direction toward learning goals, marks critical features requiring attention, controls frustration, and demonstrates concepts without eliminating the challenge.

The simultaneous reduction of GCL and ECL presents a theoretical tension. When both extraneous and germane load are reduced simultaneously, Cognitive Load Theory predicts diminished learning because less capacity is engaged in schema-building processes, even if the task feels easier. Both AI conditions reduced extraneous and germane load compared to the control group, yet all three groups showed equivalent knowledge gains from T1 to T2—not the worse learning outcomes that CLT would predict.

Several interpretations warrant consideration. The interpretation offered in the previous section—that AI assistance enabled students to encounter learning opportunities otherwise inaccessible—suggests expanded engagement might have offset reduced cognitive effort per task element: students processed more material, albeit less deeply per unit. Additionally, the 90-minute timeframe may have been insufficient for differential cognitive processing to consolidate into measurable knowledge differences. Finally, the learning assessments may have lacked sensitivity to detect differences that existed but fell below the measurement threshold.

8.3. Autonomy and motivation: how scaffolding design shapes engagement

The motivational findings reveal that how AI supports task completion matters as much as whether it provides support (Finding 3). Both tools reduced frustration without significant differences between them, yet only Iris enhanced intrinsic motivation. This divergence illuminates an interesting distinction: ChatGPT enables completion by removing the challenge (revealing solutions), while Iris supports engagement by scaffolding interaction with the challenge (offering hints without revealing answers). The difference is between making a task easy versus making it approachable.

Self-Determination Theory (SDT) provides a theoretical lens for understanding this pattern (Ryan & Deci, 2000). According to this framework, intrinsic motivation depends on satisfying three basic psychological needs: autonomy (experiencing volition and choice), competence (feeling effective), and relatedness (connection with others). While both AI tools may have supported feelings of competence by enabling task completion, they differ in their impact on autonomy. Iris's hint-based design preserved learner autonomy by withholding complete solutions and requiring students to construct their own understanding, maintaining what De Charms (1983) called an “internal locus of causality”—the sense that one's actions stem from personal volition rather than external control. ChatGPT's direct-answer approach, by contrast, eliminated autonomous problem-solving and produced no motivational benefit over traditional resources. This pattern aligns with SDT's prediction that controlling environments undermine intrinsic motivation even when they are helpful (Deci & Ryan, 1987).

This suggests that effective support in education may not be about removing all barriers but about selectively helping students overcome unnecessary obstacles while preserving those that promote growth. ChatGPT makes everyone succeed uniformly, while Iris makes everyone able to try meaningfully. This distinction is shaped by pedagogical design rather than mere technological capability.

However, the motivational advantage did not translate into a learning advantage within this study. SDT research has found that autonomy-supportive conditions produce greater conceptual learning compared to controlling conditions, even when rote recall does not differ (Grolnick & Ryan, 1987). Ryan and Deci summarize this pattern as students in controlling environments learning “less effectively, especially when learning requires conceptual, creative processing” (Ryan & Deci, 2000). Yet no group differences emerged on the Knowledge Assessment or Code Comprehension measures, both of which assessed conceptual understanding rather than rote recall.

Several boundary conditions may explain this null finding. First, the 90-minute session may have been too brief for differential motivation to influence knowledge consolidation—Grolnick and Ryan's study found effects on retention more pronounced at an 8-day follow-up. Second, concurrent programming's procedural implementation demands may differ from the text comprehension tasks where SDT's learning effects have been demonstrated. Third, the learning measures may not have captured subtle differences in understanding that our instruments were not designed to detect.

These findings also speak to Koedinger and Aleven's assistance dilemma, the pedagogical decision of how to balance providing guidance against requiring learners to generate solutions themselves (Koedinger

& Aleven, 2007). The dilemma arises because more guidance improves immediate performance while less guidance can produce more effortful processing that sometimes benefits long-term retention and transfer. This study's findings suggest the dilemma may fractionate into separable affective and cognitive dimensions that do not necessarily resolve together.

On the affective dimension, Iris appears to have navigated the dilemma successfully: it reduced frustration to the same degree as ChatGPT while uniquely enhancing intrinsic motivation. Students experienced the emotional benefits of support without the motivational costs of having challenge removed entirely. On the cognitive dimension, however, neither AI tool resolved the dilemma in favor of superior learning—both conditions produced equivalent knowledge gains to the no-AI control. This separation implies that solving the assistance dilemma for affect (reducing frustration while maintaining engagement) does not automatically solve it for cognition (optimizing for durable knowledge). Instructional designers may need to treat these as distinct optimization problems rather than assuming that motivationally effective scaffolding will also be cognitively optimal.

8.4. The perception-reality gap: the comfort trap of effortless tools

The systematic preference for ChatGPT across multiple dimensions reveals a troubling disconnect between student perceptions and pedagogical effectiveness (Finding 4). Students appear to gravitate toward tools that make tasks feel easier, yet this preference for comfort is pedagogically counterproductive. The tool students perceived as most helpful was also the one that most reduced their GCL, the very mental effort associated with building understanding. This interpretation is likewise constrained by the limited reliability of the two-item GCL subscales. Meanwhile, Iris received lower ratings on perceived helpfulness. This occurred despite Iris maintaining descriptively better cognitive engagement and significantly enhancing motivation.

This “comfort trap” reveals a fundamental challenge in educational technology: students cannot reliably judge what best supports their learning based on subjective experience alone. What feels most helpful, maximum support with minimal effort, may actually undermine the productive struggle necessary for deep learning. This finding cautions against using student satisfaction as the primary metric for evaluating educational AI tools and aligns with extensive research showing that learners are poor judges of their own learning. Deslauriers et al. (2019) demonstrated that students consistently rate fluent, passive instruction more favorably than active learning approaches, despite learning significantly less.

The preference for ChatGPT appears to stem from its ability to minimize cognitive effort, precisely the characteristic that may limit its pedagogical effectiveness. This creates tension for educators and policymakers: should student satisfaction or learning effectiveness be prioritized? The marketing pressures and student-as-customer mentality prevalent in higher education may push institutions toward adopting tools that students prefer, even when those tools may be pedagogically suboptimal.

This pattern extends research on metacognitive illusions to the domain of AI tool selection. When a tool makes tasks feel effortless, students may interpret this fluency as evidence of effective learning support, even when underlying processes that sustain engagement have been bypassed. The mechanism may run deeper: metacognitive monitoring itself might constitute a form of germane cognitive load (Klepsch & Seufert, 2020). If so, the reduced germane load observed in AI conditions may have directly impaired students' ability to accurately judge their own learning—the very tool that feels most helpful may undermine the cognitive processes needed to recognize that feeling as misleading.

This finding also highlights the importance of educating students about the learning process itself. If students understood that productive struggle enhances retention, they might make different choices about tool use. Developing students' metacognitive awareness, their

understanding of how learning works, may be as important as teaching content knowledge in the AI era.

8.5. Practical implications for educators and institutions

This study derives four actionable insights for educational practice. First, educators should carefully consider their goals when integrating AI tools. If the objective is to reduce frustration and improve course completion rates, both ChatGPT and Iris appear effective. However, if the goal is to enhance intrinsic motivation, only Iris demonstrated benefits—suggesting that scaffolded approaches better support sustained engagement. If the goal is to enhance conceptual understanding, neither tool showed advantages over traditional resources, at least in the short term as measured by this study.

Second, the findings suggest that AI tools might be most valuable for helping students overcome critical hurdles rather than as learning enhancers. For students who might otherwise struggle to complete assignments due to syntactic difficulties or debugging challenges, AI assistance could mean the difference between engagement and dropout. Notably, if the goal is to help students past the hardest obstacles while preserving productive challenge, scaffolded tools like Iris may be preferable to unrestricted AI, which tends to smooth over difficulties uniformly. This interpretation may help educators set appropriate expectations and design complementary learning activities.

Third, assessment practices and course design must evolve to maintain validity and align incentives with learning goals. In practice and formative homework, instructors can default to hint-first assistance with gated escalation, and award credit for process evidence, such as tests, diffs, and brief rationales, so the pedagogically desirable path also becomes the easiest route to points. In summative contexts, the focus should shift to code reading, debugging, explanation, and delayed or transfer items that resist superficial solution delivery. These choices align incentives with learning goals rather than with raw completion, addressing the assessment validity concerns raised by the performance-learning dissociation.

Finally, for institutions navigating AI policy, the findings suggest that scaffolded AI systems represent a defensible middle ground between outright AI bans and unrestricted access. Scaffolded tools like Iris preserve motivational benefits while avoiding the perception trap that makes unrestricted AI pedagogically concerning. The policy question shifts from “whether to permit AI” to “how to design AI that supports learning goals.”

8.6. Future directions and open questions

This study opens several avenues for future research. Longitudinal investigations could examine whether the performance-learning dissociation persists over time or whether early AI assistance influences later unassisted performance. Studies examining transfer tasks, problems requiring application of learned concepts in novel contexts, could provide deeper insights into the quality of AI-supported learning.

The theoretical findings raise questions that warrant further investigation. The fractionation of the assistance dilemma into separable affective and cognitive dimensions—where Iris resolved the dilemma for motivation but not for learning—may or may not generalize to other educational domains and longer timeframes. Similarly, Self-Determination Theory predicted that autonomy-supportive conditions would enhance conceptual learning, yet no learning differences emerged; future work should examine whether the 90-minute timeframe was too brief for such effects to manifest, or whether programming tasks differ fundamentally from the text comprehension domains where SDT's learning effects have been established. The equivalent learning outcomes despite reduced germane cognitive load also challenge Cognitive Load Theory predictions and merit further theoretical attention.

The optimal design of scaffolded AI systems remains underspecified. While Iris's approach showed motivational advantages over unrestricted AI, the critical design features remain unclear. Future research

should isolate which scaffolding techniques—guiding questions, conceptual feedback, worked examples, or solution withholding—contribute most to balancing frustration reduction with productive struggle. Understanding how scaffolding granularity affects this balance could inform the design of more effective educational AI systems. Adaptive systems that adjust support levels based on individual student needs and learning objectives represent a promising direction.

Finally, the broader implications of widespread AI adoption in education deserve careful study. As these tools become ubiquitous, fundamental questions arise about the nature of programming expertise, the goals of computer science education, and the skills students need in an AI-augmented future. Rather than viewing AI as a threat to learning, education might be reconceptualized to focus on skills that complement AI capabilities—such as problem formulation, solution evaluation, and creative design.

9. Threats to validity

Following Runeson and Höst's validity framework for empirical software engineering research (Runeson & Höst, 2009), this analysis identifies several threats to the validity of these findings that must be considered when interpreting results.

9.1. Construct validity

Construct validity concerns whether we measured what we intended to measure. A few threats arise from the assessment design employed. The knowledge assessment relied on six conceptual questions and a three-item code comprehension task, which may inadequately capture the multifaceted nature of programming understanding; the coarse granularity of the Code Comprehension measure (0–3 points) may reduce sensitivity to subtle effects. The study included no additional programming tasks or code-writing assessments that would test students' ability to apply learned concepts in novel contexts without AI support. These assessment limitations mean that null findings for learning outcomes should be interpreted cautiously. However, the combination of two learning measures (conceptual knowledge and code comprehension), objective LMS-based performance data, and validated psychological scales with acceptable-to-high internal consistency (reported in Section 4.4) provides converging evidence; the pre- and post-identical-item design increases sensitivity to within-student change; and all instruments were tightly aligned with the concurrency content.

9.2. Internal validity

Internal validity addresses whether observed relationships are causal rather than spurious. The high attrition rate (39.2 % of enrolled participants) may have introduced systematic selection bias, as students who completed the study following all the rules may differ meaningfully from those who withdrew or were excluded due to attention check failures or timing issues. The experimental context itself may have influenced behavior through Hawthorne effects, where participants modify their behavior due to awareness of being observed. However, random assignment yielded equivalent groups in terms of gender, age, and prior programming experience (Table 3); exclusion rules were pre-specified and applied uniformly across arms; attrition proportions were comparable between conditions; sessions followed a standardized, tutor-training-backed script with in-room supervision; and multiple compliance checks such as attention items and temporal-order constraints collectively mitigate validity issues.

9.3. External validity

External validity concerns the generalizability of findings beyond the studied context. While this study addresses methodological gaps in prior work through its three-arm design and controlled model comparison, generalizability remains constrained by several contextual factors: data collection at a single institution, within one introductory programming

course for management students, using a single concurrency exercise, over a 90-minute session. These limitations mirror the scope constraints identified in earlier studies. The sample was drawn from a single institution and focused specifically on Java programming and concurrent programming concepts, limiting generalizability to other programming languages, educational contexts, or student populations. In addition, the controlled setting may not perfectly reflect how students would naturally integrate these tools into their learning processes outside of research contexts. However, the sample size is large enough to provide some generalizability due to a high probability of more diverse backgrounds. Replication across diverse institutional contexts, student populations, programming topics, and timeframes is needed before conclusions can be generalized broadly.

9.4. Reliability

Reliability addresses whether the study can be replicated with consistent results. The absence of delayed testing prevents conclusions about retention or the durability of any observed learning effects. Additionally, while this paper provides detailed descriptions of the experimental protocol and materials to support replication, the rapidly evolving nature of AI tools means that exact replication with identical tool capabilities may become impossible as these systems continue to develop. However, the exercise, surveys, and LMS configuration were fixed across all 35 sessions; we document the full tutor script and procedure; and we release the used exercise, survey questions, pseudonymized dataset and analysis code, enabling computational reproducibility and facilitating conceptual replications even as specific model versions change. While ICL and GCL subscales contain only two items each, which limits traditional reliability assessment, the PCA analysis confirms that all items contribute meaningfully to the overall cognitive load construct. Results for two-item subscales should be interpreted with caution.

10. Conclusion

The pattern of reduced stress without detectable learning differences observed in this study reveals strong tensions in AI-supported programming education. The findings demonstrate that generative AI tools can improve exercise performance—with the ChatGPT Group achieving scores 2.4 times higher than the No-AI control group—while showing no significant differences in conceptual understanding. This performance-learning dissociation challenges the field to reconsider what “success” means in an era when AI can eliminate traditional barriers to task completion.

Perhaps more troubling is the comfort trap uncovered by this study: students systematically preferred ChatGPT's direct solutions over Iris's scaffolded guidance, rating it as more helpful and easier to use, despite ChatGPT offering no learning or motivational advantages. This preference for the pedagogically weaker tool exposes a critical vulnerability in student-driven adoption of educational technology. When learners cannot distinguish between what feels helpful and what actually promotes understanding, market forces may push institutions toward tools that maximize satisfaction at the expense of learning.

Yet the results also point toward a more nuanced path forward. Iris demonstrated that thoughtfully designed AI support can thread the needle between assistance and challenge—reducing frustration while uniquely enhancing intrinsic motivation. The key lies not in AI's raw capabilities but in its pedagogical implementation: scaffolded hints that preserve student agency are more educationally valuable than unrestricted solution access, even when students perceive them as less helpful. This finding underscores that the question is not whether to use AI in education, but how to design AI systems that support genuine learning rather than merely enabling task completion.

As programming education stands at this crossroads, this study offers both warnings and guideposts. The warnings are clear: AI tools that maximize performance and minimize effort may create an illusion of learning that satisfies students while showing no statistical improvement

in understanding. The guideposts are also clear: carefully calibrated AI support can reduce unnecessary frustration without eliminating productive struggle, and may even enhance student motivation when designed to preserve challenge and autonomy.

Moving forward, educators must resist the siren song of easy solutions—both for themselves and their students. Assessment practices must evolve to maintain validity when performance no longer indicates understanding. Students need education about learning itself, understanding why difficulty and effort, not ease and fluency, drive genuine comprehension. And developers of educational AI must prioritize pedagogical effectiveness over user satisfaction metrics.

The future of programming education in an AI-saturated world need not be dystopian. The findings suggest these tools may help struggling students engage with material that might otherwise prove too challenging to complete. But realizing this potential requires acknowledging an uncomfortable truth: the AI tools students want may not be the ones they need. The challenge ahead is crafting educational experiences that harness AI's power to reduce unnecessary barriers while preserving the essential difficulties that make learning possible.

CRediT authorship contribution statement

Patrick Bassner: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Data curation, Conceptualization. **Ben Lenk-Ostendorf:** Writing – review & editing, Visualization, Data curation, Methodology, Investigation. **Ramona Beinstingel:** Writing – review & editing, Writing – original draft, Methodology, Investigation. **Tobias Wasner:** Writing – review & editing. **Stephan Krusche:** Writing – review & editing, Writing – original draft, Supervision, Resources, Methodology, Funding acquisition, Conceptualization.

Ethics approval

This study was approved by the Ethics Committee of the Technical University of Munich (Approval ID: 2025–153-NM-BA). Informed consent was obtained from all participants prior to participation, and their privacy rights were strictly observed throughout the research process. Data were pseudonymized before analysis, with only the merging researcher having access to identifiable information. All personal data can be obtained by sending a request e-mail to the corresponding author.

Declaration of competing interest

The authors declare no competing financial or personal interests that could have influenced the work reported in this article.

Acknowledgments

During the preparation of this work, the authors used ChatGPT (4.0, 5) and Claude Sonnet (4, 4.1) and Opus (4, 4.1) to improve readability, grammar, and clarity throughout the manuscript. They reviewed and edited all AI-improved text to ensure accuracy and consistency.

Data availability

The anonymized dataset, the analysis code, the used programming assignment and the survey questions are available at Zenodo: <https://doi.org/10.5281/zenodo.16905779>

References

- Aleven, V., Roll, I., McLaren, B. M., & Koedinger, K. R. (2016). Help helps, but only so much: Research on help seeking with intelligent tutoring systems. *International Journal of Artificial Intelligence in Education*, 26(1), 205–223. <https://doi.org/10.1007/s40593-015-0089-1>
- Baker, R. S., Corbett, A. T., & Koedinger, K. R. (2004). Detecting student misuse of intelligent tutoring systems. In D. Hutchison, T. Kanade, J. Kittler, J. M. Kleinberg, F. Mattern, J. C. Mitchell, M. Naor, O. Nierstrasz, C. P. Rangan, B. Steffen, M. Sudan, D. Terzopoulos, D. Tygar, M. Y. Vardi, G. Weikum, J. C. Lester, R. M. Vicari,

- & F. Paraguacu (Eds.), *Intelligent tutoring systems* (vol. 3220, pp. 531–540). Berlin, Heidelberg: Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-30139-4_50
- Bassner, P., Frankford, E., & Krusche, S. (2024). Iris: An AI-driven virtual tutor for computer science education. In *Proceedings of the 2024 innovation and technology in Computer science education V* (Vol. 1). Milan, Italy. <https://doi.org/10.1145/3649217.3653543>
- Bastani, H., Bastani, O., Sungu, A., Ge, H., Kabakci, Ö., & Mariman, R. (2025). Generative AI without guardrails can harm learning: Evidence from high school Mathematics. *Proceedings of the National Academy of Sciences*, 122(26), e2422633122. <https://doi.org/10.1073/pnas.2422633122>
- Bjork, R. A. (1994). Memory and metamemory considerations in the training of human beings. In J. Metcalfe, & A. P. Shimamura (Eds.), *Metacognition* (pp. 185–206). The MIT Press. <https://doi.org/10.7551/mitpress/4561.003.0011>
- Bjork, E. L., & Bjork, R. A. (2011). Making things hard on yourself, but in a good way: Creating desirable difficulties to enhance learning. In *Psychology and the real World: Essays illustrating fundamental contributions to society* (pp. 56–64). Worth Publishers.
- Cohen, J. (1988). *Statistical power analysis for the behavioral sciences*. Routledge. <https://doi.org/10.4324/9780203771587>
- D'Mello, S., & Graesser, A. (2012). Dynamics of affective states during complex learning. *Learning and Instruction*, 22(2), 145–157. <https://doi.org/10.1016/j.learninstruc.2011.10.001>
- De Charms, R. (1983). *Personal causation: The internal affective determinants of behavior*. Routledge.
- Deci, E. L., & Ryan, R. M. (1987). The support of autonomy and the control of behavior. *Journal of Personality and Social Psychology*, 53(6), 1024–1037. <https://doi.org/10.1037/0022-3514.53.6.1024>
- Delacre, M., Leys, C., Mora, Y. L., & Lakens, D. (2019). Taking parametric assumptions seriously: Arguments for the use of welch's *phf*-test instead of the classical *phf*-test in one-way ANOVA. *International Review of Social Psychology*, 32(1), 13. <https://doi.org/10.5334/irsp.198>
- Denny, P., MacNeil, S., Savelka, J., Porter, L., & Luxton-Reilly, A. (2024). Desirable characteristics for AI teaching assistants in programming education. In *Proceedings of the 2024 on innovation and technology in computer science education V* (Vol. 1, pp. 408–414). Milan, Italy: ACM. <https://doi.org/10.1145/3649217.3653574>
- Denny, P., Prather, J., Becker, B. A., Finnie-Ansley, J., Hellas, A., Leinonen, J., Luxton-Reilly, A., Reeves, B. N., Santos, E. A., & Sarsa, S. (2024). Computing education in the era of generative AI, *commun. ACM*, 67(2), 56–67. <https://doi.org/10.1145/3624720>
- Deslauriers, L., McCarty, L. S., Miller, K., Callaghan, K., & Kestin, G. (2019). Measuring actual learning versus feeling of learning in response to being actively engaged in the classroom. *Proceedings of the National Academy of Sciences*, 116(39), 19251–19257. <https://doi.org/10.1073/pnas.1821936116>
- Fan, G., Liu, D., Zhang, R., & Pan, L. (2025). The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance: A comparative study with traditional pair programming and individual approaches. *International Journal of STEM Education*, 12(1), 16. <https://doi.org/10.1186/s40594-025-00537-3>
- Ferdowsi, K., Huang, R. L., James, M. B., Polikarpova, N., & Lerner, S. (2024). Validating AI-generated code with live programming. In *Proceedings of the CHI conference on Human factors in computing systems* (pp. 1–8). Honolulu, HI, USA: ACM. <https://doi.org/10.1145/3613904.3642495>
- France, S. L. (2024). Navigating software development in the Chatgpt and Github copilot era. *Business Horizons*, 67(5), 649–661. <https://doi.org/10.1016/j.bushor.2024.05.009>
- Games, P. A., & Howell, J. F. (1976). Pairwise multiple comparison procedures with unequal N's and/or variances: A monte carlo study. *Journal of Educational Statistics*, 1(2), 113–125. <https://doi.org/10.3102/10769986001002113>
- Gao, L., Ma, X., Lin, J., & Callan, J. (2023). Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st annual meeting of the Association for Computational Linguistics (volume 1: Long papers)*. Toronto, Canada: Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.acl-long.99>
- Gardella, N., Pettit, R., Riggs, S. L., & Performance, W. (2024). Emotion, and self-efficacy of novice programmers using AI code generation. In *Proceedings of the 2024 on innovation and technology in computer science education V* (Vol. 1, pp. 290–296). Milan, Italy: ACM. <https://doi.org/10.1145/3649217.3653615>
- Grinschgl, S., Papenmeier, F., & Meyerhoff, H. S. (2021). Consequences of cognitive offloading: Boosting performance but diminishing memory. *Quarterly Journal of Experimental Psychology*, 74(9), 1477–1496. <https://doi.org/10.1177/17470218211008060>
- Grolnick, W. S., & Ryan, R. M. (1987). Autonomy in children's learning: An experimental and individual difference investigation. *Journal of Personality and Social Psychology*, 52(5), 890–898. <https://doi.org/10.1037/0022-3514.52.5.890>
- Horia Alexandru, M. (2025). *LLM intelligent agent tutoring in higher education courses using a RAG approach* (pp. 589–599). Cham: Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-83520-9_54
- Huang, A. Y. Q., Lin, C.-Y., Su, S.-Y., & Yang, S. J. H. (2025). The impact of genai-enabled coding hints on students' programming performance and cognitive load in an SRL-based Python course. *British Journal of Educational Technology*, <https://doi.org/10.1111/bjet.13589>
- Jennings, J., & Muldner, K. (2021). When does scaffolding provide too much assistance? A code-tracing tutor investigation. *International Journal of Artificial Intelligence in Education*, 31(4), 784–819. <https://doi.org/10.1007/s40593-020-00217-z>
- Kalyuga, S. (2007). Expertise reversal effect and its implications for learner-tailored instruction. *Educational Psychology Review*, 19(4), 509–539. <https://doi.org/10.1007/s10648-007-9054-3>

- Karabenick, S. A. (2004). Perceived achievement goal structure and college student help seeking. *Journal of Educational Psychology*, 96(3), 569–581. <https://doi.org/10.1037/0022-0663.96.3.569>
- Kasnezi, E., Sessler, K., Kuchemann, S., Bannert, M., Dementieva, D., Fischer, F., Gasser, U., Groh, G., Günemann, S., Hüllermeier, E., Krusche, S., Kutyniok, G., Michaeli, T., Nerdel, C., Pfeffer, J., Poquet, O., Sailer, M., Schmidt, A., Seidel, T. ... Kasnezi, G. (2023). Chatgpt for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103, 102274. <https://doi.org/10.1016/j.lindif.2023.102274>
- Kazemitabaar, M., Ye, R., Wang, X., Henley, A. Z., Denny, P., Craig, M., & Grossman, T. (2024). CodeAid: Evaluating a classroom deployment of an LLM-based programming assistant that balances student and educator needs. In *Proceedings of the CHI conference on Human factors in computing systems* (pp. 1–20). Honolulu, HI, USA: ACM. <https://doi.org/10.1145/3613904.3642773>
- Kirova, V. D., Ku, C. S., Laracy, J. R., & Marlowe, T. J. (2024). Software engineering education must adapt and evolve for an LLM environment. In *Proceedings of the 55th ACM Technical Symposium on Computer science education V* (Vol. 1, pp. 666–672). Portland, OR, USA: ACM. <https://doi.org/10.1145/3626252.3630927>
- Klepsch, M., Schmitz, F., & Seufert, T. (2017). Development and validation of two instruments measuring intrinsic, extraneous, and germane cognitive load. *Frontiers in Psychology*, 8, 1997. <https://doi.org/10.3389/fpsyg.2017.01997>
- Klepsch, M., & Seufert, T. (2020). Understanding instructional design effects by differentiated measurement of intrinsic, extraneous, and germane cognitive load. *Instructional Science*, 48(1), 45–77. <https://doi.org/10.1007/s11251-020-09502-9>
- Klepsch, M., & Seufert, T. (2021). Making an effort versus experiencing load. *Frontiers in Education*, 6, 645284. <https://doi.org/10.3389/feduc.2021.645284>
- Koedinger, K. R., & Alevan, V. (2007). Exploring the assistance dilemma in experiments with cognitive tutors. *Educational Psychology Review*, 19(3), 239–264. <https://doi.org/10.1007/s10648-007-9049-0>
- Krusche, S., & Seitz, A. (2018). Artemis: An automatic assessment management system for interactive learning. In *Proceedings of the 49th Technical Symposium on Computer Science Education (SIGCSE)* (pp. 284–289). ACM. <https://doi.org/10.1145/3159450.3159602>
- Kruskal, W. H., & Wallis, W. A. (1952). Use of ranks in one-criterion variance analysis. *Journal of the American Statistical Association*, 47(260), 583–621. <https://doi.org/10.1080/01621459.1952.10483441>
- Lahtinen, E., Ala-Mutka, K., & Järvinen, H.-M. (2005). A study of the difficulties of novice programmers. In *Proceedings of the 10th annual SIGCSE Conference on Innovation and Technology in Computer Science Education, iticse '05, Association for Computing Machinery* (pp. 14–18). New York, NY, USA. <https://doi.org/10.1145/1067445.1067453>
- Lakens, D. (2013). Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for t-tests and anovas. *Frontiers in Psychology*, 4, <https://doi.org/10.3389/fpsyg.2013.00863>
- Levene, H. (1960). *Robust tests for equality of variances* (pp. 278–292). Palo Alto: Contributions to Probability and Statistics, Stanford University Press.
- Li, Z., Wang, Z., Wang, W., Hung, K., Xie, H., & Wang, F. L. (2025). Retrieval-augmented generation for educational application: A systematic survey. *Computers and Education: Artificial Intelligence*, 8, 100417. <https://doi.org/10.1016/j.caeai.2025.100417>
- Liffiton, M., Sheese, B. E., Savelka, J., & Denny, P. (2023). Codehelp: Using large language models with guardrails for scalable support in programming classes. In *Proceedings of the 23rd Koli calling international conference on computing education research* (pp. 1–11). Koli, Finland: ACM. <https://doi.org/10.1145/3631802.3631830>
- Ma, X., Zhang, X., Pradeep, R., & Lin, J. (2023). Zero-Shot Listwise Document Reranking with a Large Language Model. <https://doi.org/10.48550/ARXIV.2305.02156>
- Okonkwo, C. W., & Ade-Ibajola, A. (2021). Chatbots applications in education: A systematic review. *Computers and Education: Artificial Intelligence*, 2, 100033. <https://doi.org/10.1016/j.caeai.2021.100033>
- Paas, F., Renkl, A., & Sweller, J. (2003). Cognitive load theory and instructional design: Recent developments. *Educational Psychologist*, 38(1), 1–4. https://doi.org/10.1207/S15326985EP3801_1
- Paranjape, B., Lundberg, S., Singh, S., Hajishirzi, H., Zettlemoyer, L., & Ribeiro, M. T. (2023). ART: Automatic multi-step reasoning and tool-use for large language models. <https://doi.org/10.48550/ARXIV.2303.09014>
- Peng, S., Kalliamvakou, E., Cihon, P., & Demirel, M. (2023). The impact of AI on developer productivity: evidence from GitHub copilot. <https://doi.org/10.48550/ARXIV.2302.06590>
- Prather, J., Reeves, B. N., Denny, P., Becker, B. A., Leinonen, J., Luxton-Reilly, A., Powell, G., Finnie-Ansley, J., & Santos, E. A. (2023). 's weird that it knows what i want': Usability and interactions with copilot for novice programmers. *ACM Transactions on Computer-Human Interaction*, 31(1), 1–31. <https://doi.org/10.1145/3617367>
- Rashkovits, R., & Lavy, I. (2020). Novice programmers' coping with multi-threaded software design. *Journal of Information Technology Education: Innovations in Practice*, 19, 75–89. <https://doi.org/10.28945/4609>
- Rawat, M., Gupta, A., Gooner, R., Di Bari, A., Gupta, N., & Pieraccini, R. (2025). Pre-Act: Multi-Step Planning and Reasoning Improves Acting in LLM Agents. <https://doi.org/10.48550/ARXIV.2505.09970>
- Richardson, J. T. (2011). ETA squared and partial eta squared as measures of effect size in educational research. *Educational Research Review*, 6(2), 135–147. <https://doi.org/10.1016/j.edurev.2010.12.001>
- Roll, I., Alevan, V., McLaren, B. M., & Koedinger, K. R. (2011). Improving students' help-seeking skills using metacognitive feedback in an intelligent tutoring system. *Learning and Instruction*, 21(2), 267–280. <https://doi.org/10.1016/j.learninstruc.2010.07.004>
- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- Ryan, R. M. (1982). Control and information in the intrapersonal sphere: An extension of cognitive evaluation theory. *Journal of Personality and Social Psychology*, 43(3), 450–461. <https://doi.org/10.1037/0022-3514.43.3.450>
- Ryan, A. M., & Pintrich, P. R. (1997). Should I ASK for help?" The role of motivation and attitudes in adolescents' help seeking in math class. *Journal of Educational Psychology*, 89(2), 329–341. <https://doi.org/10.1037/0022-0663.89.2.329>
- Ryan, R. M., & Deci, E. L. (2000). Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. *American Psychologist*, 55(1), 68–78. <https://doi.org/10.1037/0003-066X.55.1.68>
- Shan, R. (2025). *Retrieval-augmented generation architecture framework: Harnessing the power of RAG* (pp. 88–104). Cham: Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-77954-1_6
- Shihab, M. I. H., Hundhausen, C., Tariq, A., Haque, S., Qiao, Y., & Mulanda, B. W. (2025). The effects of Github copilot on computing students' programming effectiveness, efficiency, and processes in brownfield coding tasks. In *Proceedings of the 2025 ACM Conference on International Computing Education Research V.1, ICER '25, Association for Computing Machinery* (pp. 407–420). New York, NY, USA. <https://doi.org/10.1145/3702652.3744219>
- Slamecka, N. J., & Graf, P. (1978). The generation effect: Delineation of a phenomenon. *Journal of Experimental Psychology: Human Learning and Memory*, 4(6), 592–604. <https://doi.org/10.1037/0278-7393.4.6.592>
- Sweller, J., Van Merriënboer, J. J. G., & Paas, F. G. W. C. (1998). Cognitive architecture and instructional design. *Educational Psychology Review*, 10(3), 251–296. <https://doi.org/10.1023/A:1022193728205>
- Tukey, J. W. (1953). *The problem of multiple comparisons*. Chapman and Hall.
- Uttl, B., White, C. A., & Gonzalez, D. W. (2017). Meta-analysis of faculty's teaching effectiveness: Student evaluation of teaching ratings and student learning are not related. *Studies in Educational Evaluation*, 54, 22–42. <https://doi.org/10.1016/j.stueduc.2016.08.007>
- van de Pol, J., Volman, M., & Beishuizen, J. (2010). Scaffolding in teacher–student interaction: A decade of research. *Educational Psychology Review*, 22(3), 271–296. <https://doi.org/10.1007/s10648-010-9127-6>
- Van Merriënboer, J. J. G., & Sweller, J. (2005). Cognitive load theory and complex learning: Recent developments and future directions. *Educational Psychology Review*, 17(2), 147–177. <https://doi.org/10.1007/s10648-005-3951-0>
- Vygotsky, L. S. (1978). *Mind in society: Development of higher psychological processes*. Cambridge, MA: Harvard University Press.
- Welch, B. L. (1951). On the comparison of several mean values: an alternative approach. *Biometrika*, 38 (3/4), 330. <https://doi.org/10.2307/2332579>
- Wood, D., Bruner, J. S., & Ross, G. (1976). The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2), 89–100. <https://doi.org/10.1111/j.1469-7610.1976.tb00381.x>
- Wu, J., Zhu, J., & Liu, Y. (2025). Agentic Reasoning: Reasoning LLMs with Tools for the Deep Research. <https://doi.org/10.48550/ARXIV.2502.04644>
- Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., Zheng, R., Fan, X., Wang, X., Xiong, L., Zhou, Y., Wang, W., Jiang, C., Zou, Y., Liu, X. ... Gui, T. (2025). The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2), <https://doi.org/10.1007/s11432-024-4222-0>
- Yan, V. X., Bjork, E. L., & Bjork, R. A. (2016). On the difficulty of mending metacognitive illusions: A priori theories, fluency effects, and misattributions of the interleaving benefit. *Journal of Experimental Psychology: General*, 145(7), 918–933. <https://doi.org/10.1037/xge0000177>